TAD – Listas Lineares

Prof. Henrique Batista da Silva

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Listas Lineares

- Uma das formas mais simples de interligar os elementos de um conjunto.
- Estrutura em que as operações inserir, retirar e localizar são definidas.
- São estruturas flexíveis:
  - podem crescer ou diminuir de tamanho durante a execução de um programa.
- Itens de uma lista podem ser:
  - Acessados;
  - Inseridos;
  - retirados.

Listas Lineares

- Duas listas podem ser concatenadas para formar uma lista única, ou uma pode ser partida em duas ou mais listas.
- As listas são adequadas quando não é possível prever a demanda por memória, permitem:
  - Manipulação de quantidades imprevisíveis de dados.
  - Manipulação de dados de formato também imprevisível.

Listas Lineares

- São úteis em aplicações tais como manipulação simbólica, gerência de memória, simulação e compiladores.

- Uma lista linear é uma sequência de zero ou mais itens $x_1, x_2, \ldots, x_n$, na qual x_i é de uma determinado tipo e n representa o tamanho da lista linear.

Listas Lineares

- Sua principal propriedade estrutural envolve as posições relativas dos itens em uma dimensão.
  - Assumindo $n \geq 1$, x_1 o primeiro item da lista e x_n é o último item da lista.
  - x_i precede x_{i+1} para $i = 1,2, \dots, n-1$
  - x_i sucede x_{i-1} para $i = 2,3, \dots, n$
  - o elemento x_i é dito estar na i-ésima posição da lista.

Operação sobre Listas Lineares

- Para criar uma TAD Lista, precisa-se definir um conjunto de operação.
- O conjunto de operações a ser definido depende de cada aplicação.
- No entanto, há um conjunto de operações necessário a uma maioria de aplicações, entre elas:
  - Criar uma lista linear vazia
  - Inserir um novo item imediatamente após o i-ésimo item.
  - Retirar o i-ésimo item.
  - Localizar o i-ésimo item para examinar e/ou alterar o conteúdo de seus componentes.
  - Fazer uma cópia da lista linear.
  - Ordenar os itens da lista em ordem ascendente ou descendente, de acordo com alguns de seus componentes.
  - Pesquisar a ocorrência de um item com um valor particular em algum componente.

Representação de Listas Lineares

- Existem várias estruturas de dados que podem ser usadas para representar listas lineares:
- As duas representações mais utilizadas são:
  - por meio de arranjos; e
  - estruturas auto referenciadas (ou apontadores para linguagens como C e C++).

Representação de Listas Lineares

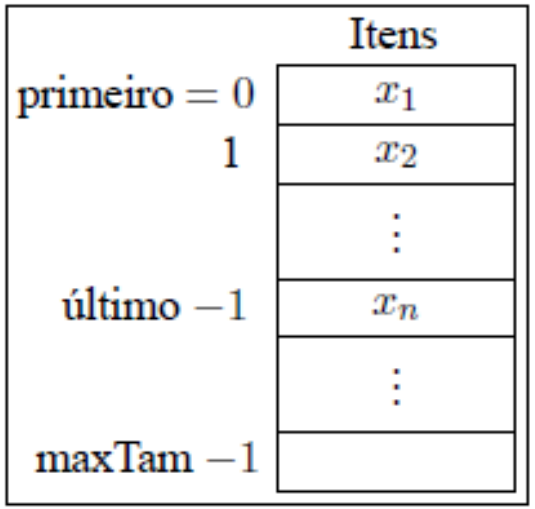
- Exemplos de Operações:
  - Lista(maxTam). Cria uma lista vazia.
  - insere(x). Insere x após o último item da lista.
  - retira(x). Retorna o item x que está na posição p da lista, retirando-o da lista e deslocando os itens a partir da posição $p + 1$ para as posições anteriores.
  - vazia(). Esta função retorna *true* se lista vazia; senão retorna *false*.
  - imprime(). Imprime os itens da lista na ordem de ocorrência.

Implementação de Listas por meio de Arranjos

- Os itens são armazenados em um arranjo de tamanho suficiente para armazenar a lista.
- Os itens da lista são armazenados em posições contíguas de memória.
- A lista pode ser percorrida em qualquer direção.
- Inserção de um novo item:
  - após o último item com custo constante.
  - no meio da lista requer um deslocamento de todos os itens localizados após o ponto de inserção.

Implementação de Listas por meio de Arranjos

- Retirar um item do início da lista requer um deslocamento de itens para preencher o espaço deixado vazio.



Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Propriedade das Listas Lineares

Insere

X_1	X_2	X_3	X_4	X_5						

prim ultim

A inserção padrão dos elementos na lista é na posição último. Porém, é possível também fazer inserção em qualquer posição.

Propriedade das Listas Lineares

Insere

X_1	X_2	X_3	X_4	X_5	X_6					

prim ultim

A inserção padrão dos elementos na lista é na posição último. Porém, é possível também fazer inserção em qualquer posição.

Propriedade das Listas Lineares

Insere

X_1	X_2	X_3	X_4	X_5	X_6	X_7				

prim ultim

A inserção padrão dos elementos na lista é na posição último. Porém, é possível também fazer inserção em qualquer posição.

Propriedade das Listas Lineares

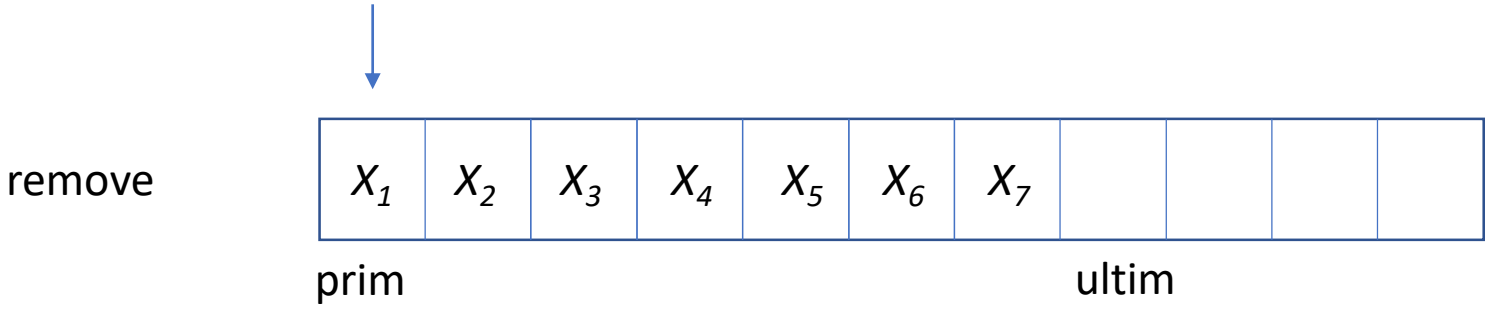
remove

X_1	X_2	X_3	X_4	X_5	X_6	X_7				

prim ultim

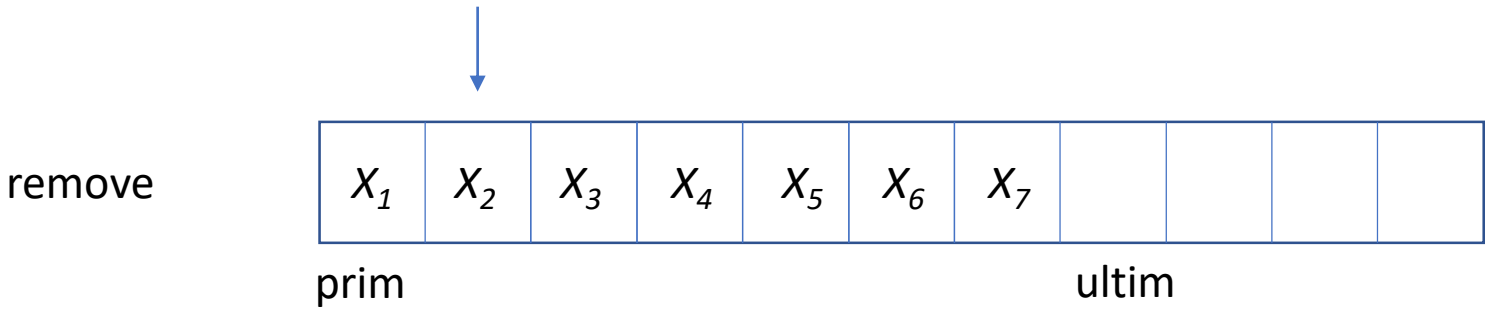
Remover: X_3

Propriedade das Listas Lineares



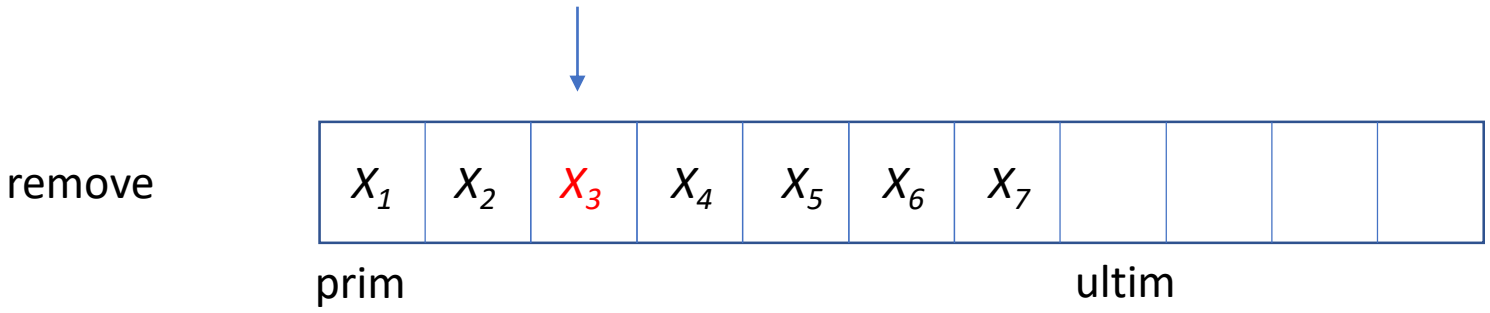
Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



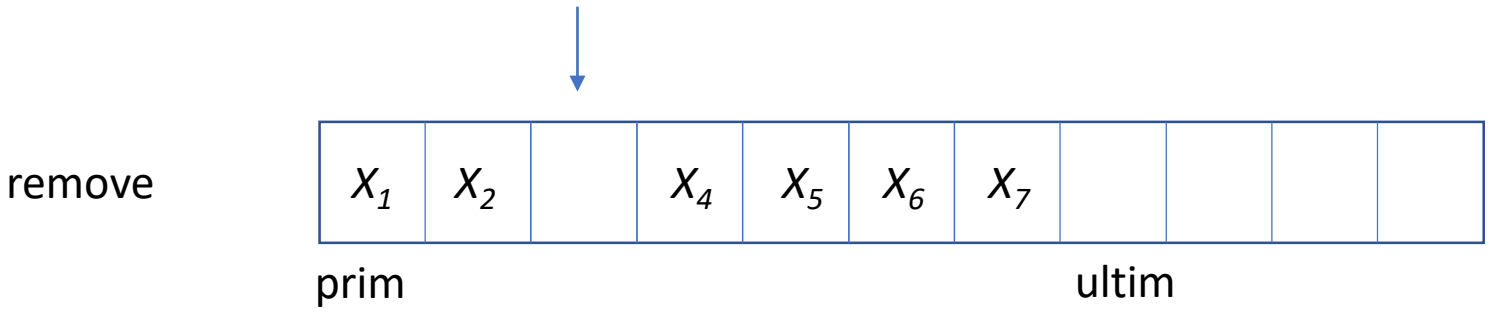
Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



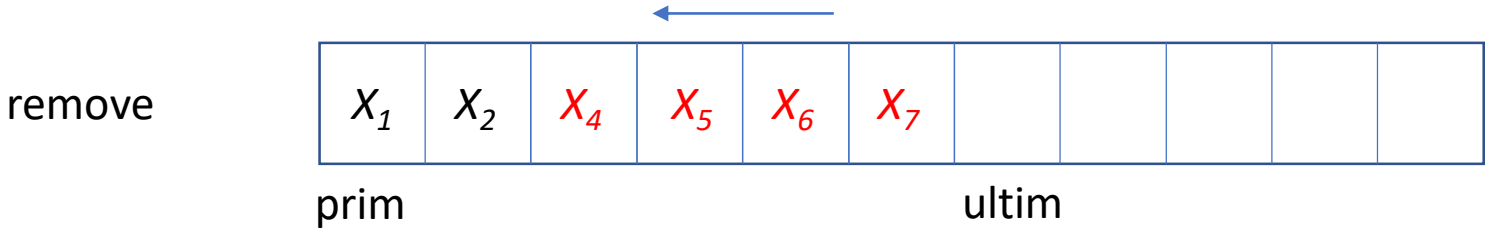
Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



Remover: X_3

Ao encontrar, remover e deslocar todos os demais elementos

Listas por meio de Arranjos

```
public class Lista {
  private Object item[];
  private int primeiro, ultimo, pos;

  public Lista () {
    this.item = new Object[1000]; this.pos = -1;
    this.primeiro = 0; this.ultimo = this.primeiro;
  }
  public Lista (int maxTam) {
    this.item = new Object[maxTam]; this.pos = -1;
    this.primeiro = 0; this.ultimo = this.primeiro;
  }
  public Object pesquisa (Object chave) {
    if (this.vazia () || chave == null) return null;
    for (int p = 0; p < this.ultimo; p++)
      if (this.item[p].equals (chave)) return this.item[p];
    return null;
  }
```

Tipo genérico do Java

Fazem referência ao início e fim da lista.

Cria um lista vazia com tamanho fixo. Observe que “primeiro” e “ultimo são iguais”

Cria um lista vazia com tamanho definido pelo usuário

Pesquisa um elemento na lista se a mesma não estiver vazia. Observe que basta percorrer a lista da posição zero até a posição do ultimo elementos da lista.

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Listas por meio de Arranjos

```
public void insere (Object x) throws Exception {
    if (this.ultimo >= this.item.length)
      throw new Exception ("Erro: A lista esta cheia");
    else { this.item[this.ultimo] = x; this.ultimo = this.ultimo + 1; }
  }
  public Object retira (Object chave) throws Exception {
    if (this.vazia () || chave == null)
      throw new Exception ("Erro : A lista esta vazia");

    int p = 0;
    while (p < this.ultimo && !this.item[p].equals (chave)) p++;

    if (p >= this.ultimo) return null;

    Object item = this.item[p]; this.ultimo = this.ultimo - 1;

    for (int aux = p; aux < this.ultimo; aux++)
      this.item[aux] = this.item[aux + 1];

    return item;
  }
```

Insere um novo elementos se a lista não estiver cheia na posição “ultimo” e atualiza o ultimo para referenciar a primeira posição livre da lista.

Na remoção é necessário pesquisa se o elemento existe. Se encontrar o elemento, ele será removido e todos os elementos posteriores a ele são deslocados para não deixar o vetor com posições vazias.

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Listas por meio de Arranjos

```
public Object retiraPrimeiro () throws Exception {
    if (this.vazia ()) throw new Exception ("Erro : A lista esta vazia");
    Object item = this.item[0]; this.ultimo = this.ultimo - 1;
    for (int aux = 0; aux < this.ultimo; aux++)
      this.item[aux] = this.item[aux + 1];
    return item;
  }
  public boolean vazia () { return (this.primeiro == this.ultimo); }
  public void imprime () {
    for (int aux = this.primeiro; aux < this.ultimo; aux++)
      System.out.println (this.item[aux].toString ());
  }
  public Object primeiro () {
    this.pos = -1; return this.proximo ();
  }
  public Object proximo () {
    this.pos++;
    if (this.pos >= this.ultimo) return null;
    else return this.item[this.pos];
  }
}
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Listas por meio de Arranjos

- Vantagem:
  - Economia de memória (os apontadores são implícitos nesta estrutura).
- Desvantagens:
  - Custo para inserir ou retirar itens da lista
    - No pior caso: deslocamento de todos os itens.
  - Em aplicações em que não existe previsão sobre o crescimento da lista
    - Exige a realocação de memória.

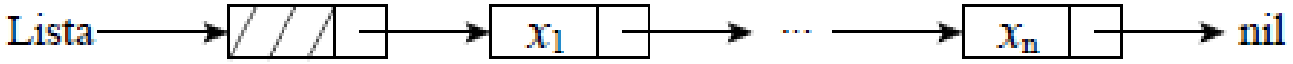
Implementação de Lista por meio de Estruturas Auto Referenciadas

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Também podem ser chamadas de listas encadeadas ou listas ligadas.
- Cada item da lista contém a informação que é necessária para alcançar o próximo item.
- Permite utilizar posições não contíguas de memória.
- Inserir e retirar elementos:
  - Não há necessidade de deslocar os itens seguintes da lista.

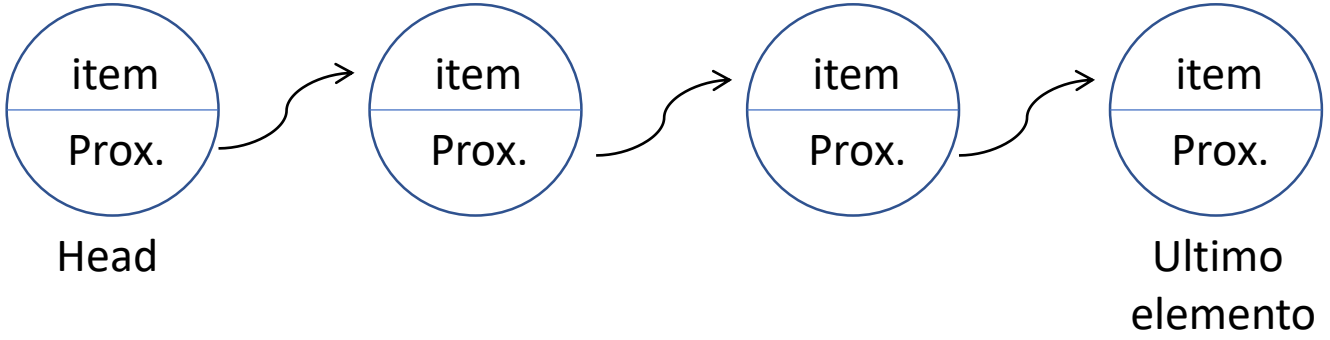
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Há uma célula cabeça (*head*):
  - simplifica as operações sobre a lista



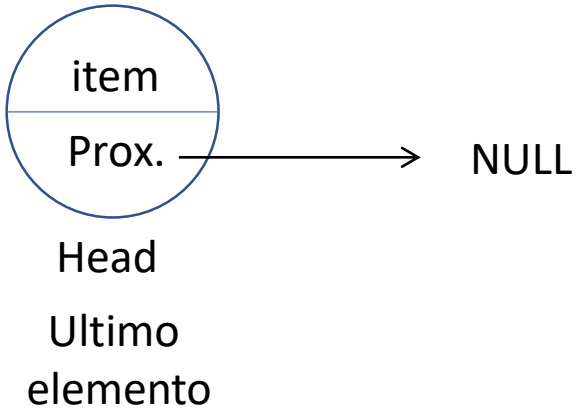
Implementação de Listas por meio de Estruturas Auto Referenciadas

- A lista é constituída de células:
  - Cada uma contém um item da lista e uma referência para a célula seguinte.



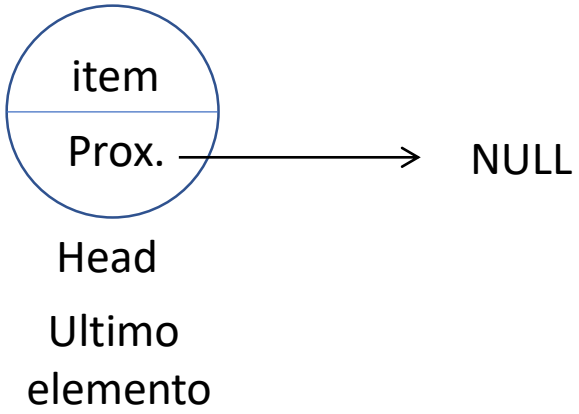
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Construção de uma Lista vazia:



Implementação de Listas por meio de Estruturas Auto Referenciadas

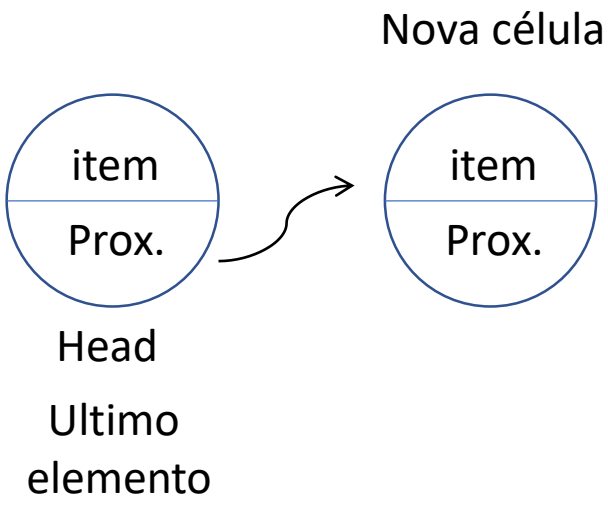
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

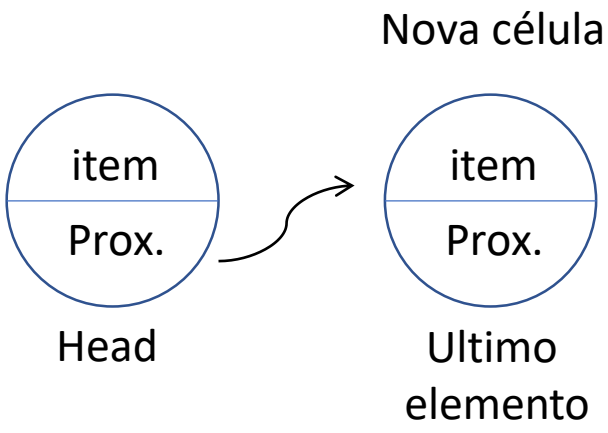
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

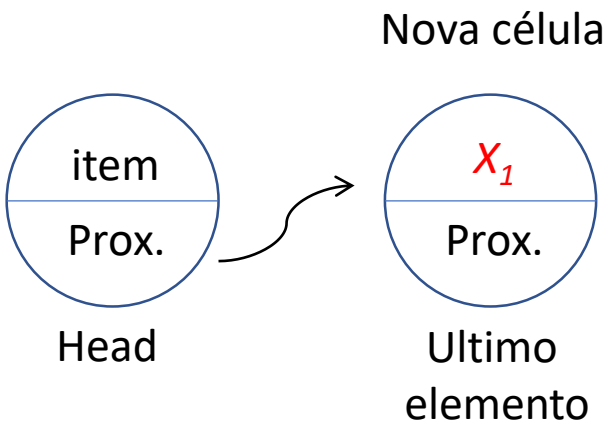
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

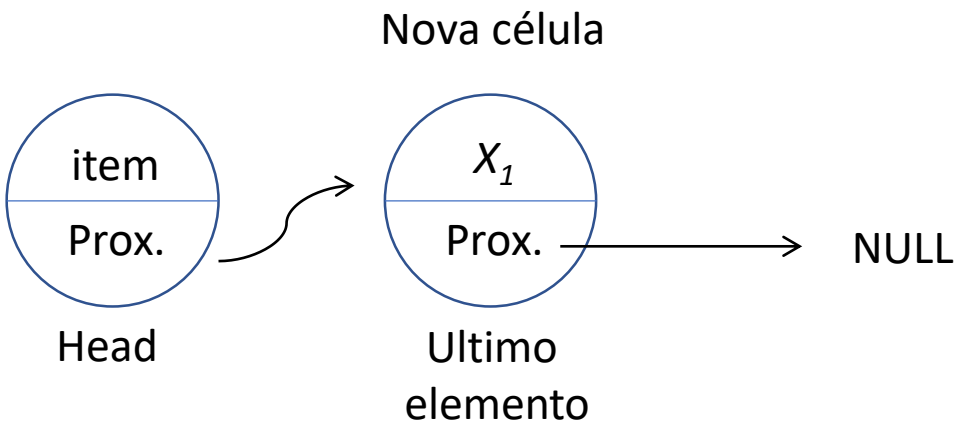
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

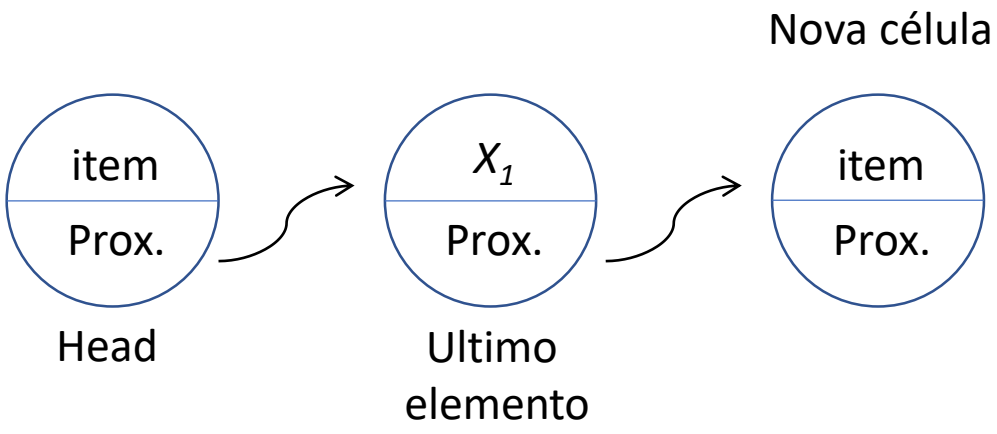
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

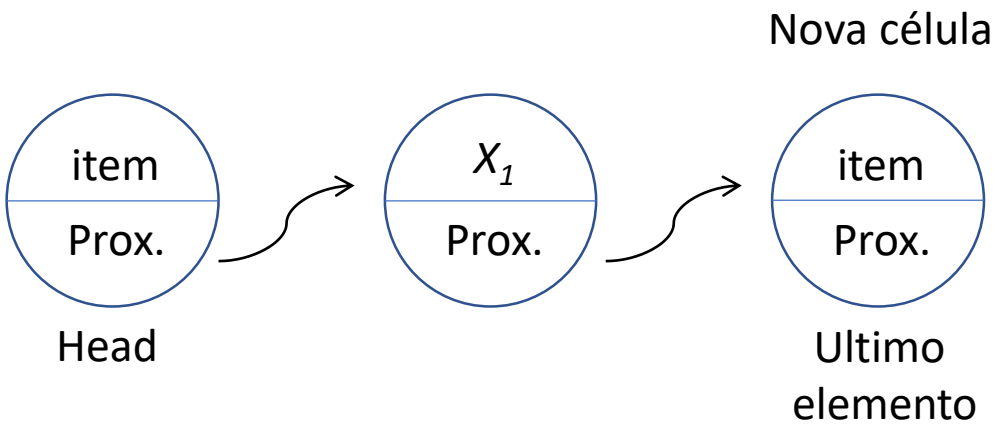
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

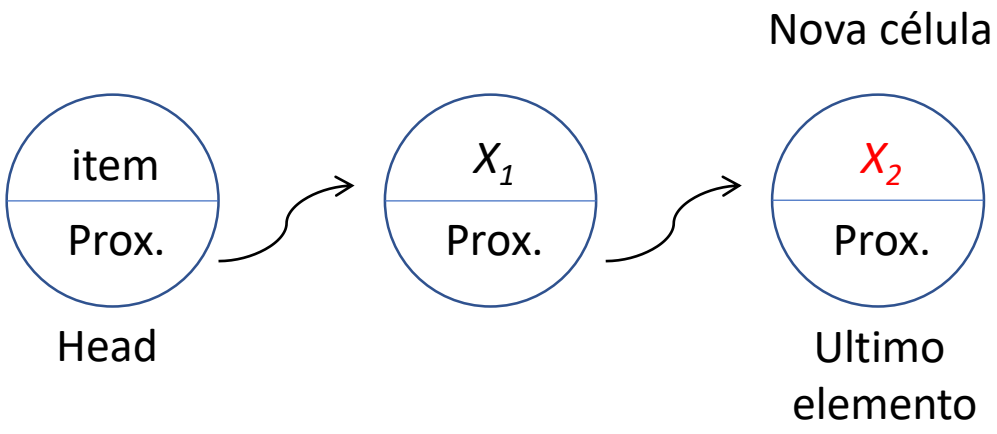
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

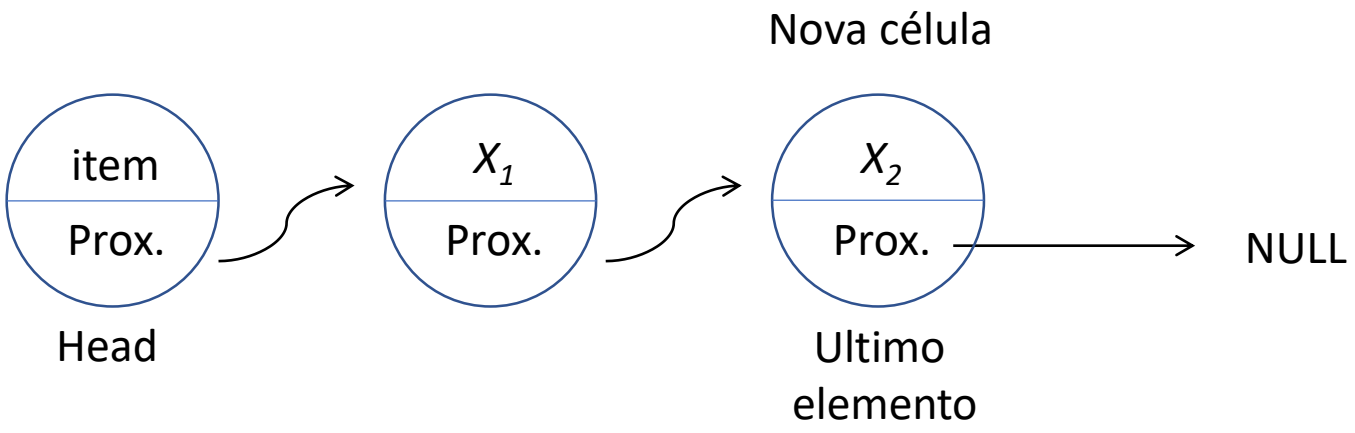
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

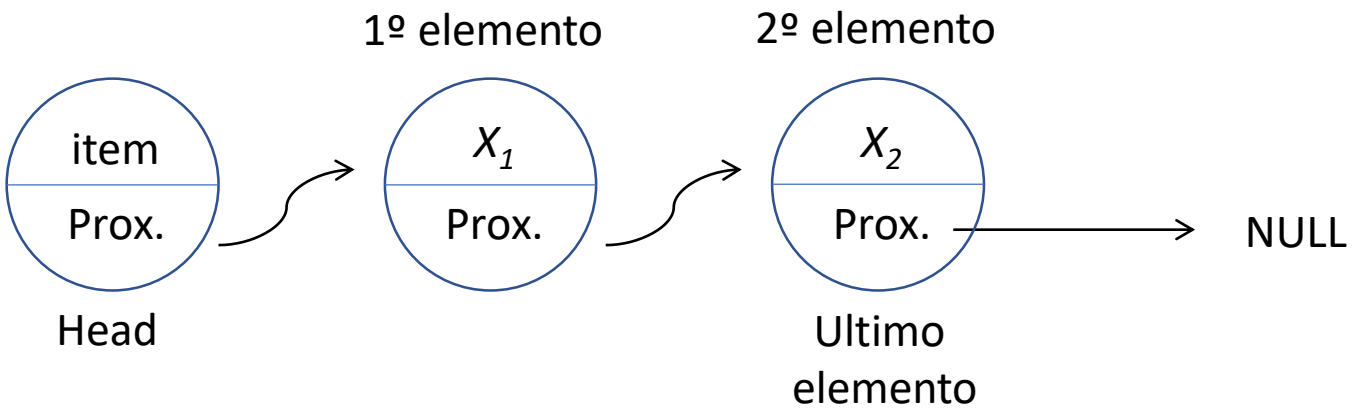
- Inserção de um novo elemento na lista:




```
public void insere (Object x) {
    this.ultimo.prox = new Celula ();
    this.ultimo = this.ultimo.prox;
    this.ultimo.item = x;
    this.ultimo.prox = null;
  }
```

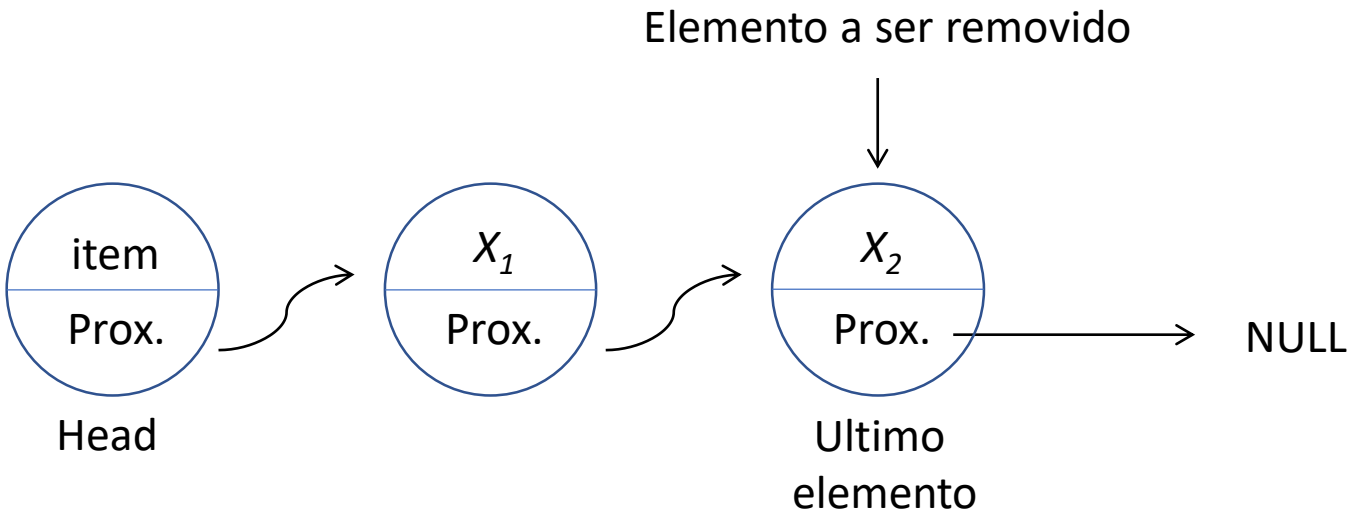
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista:



Implementação de Listas por meio de Estruturas Auto Referenciadas

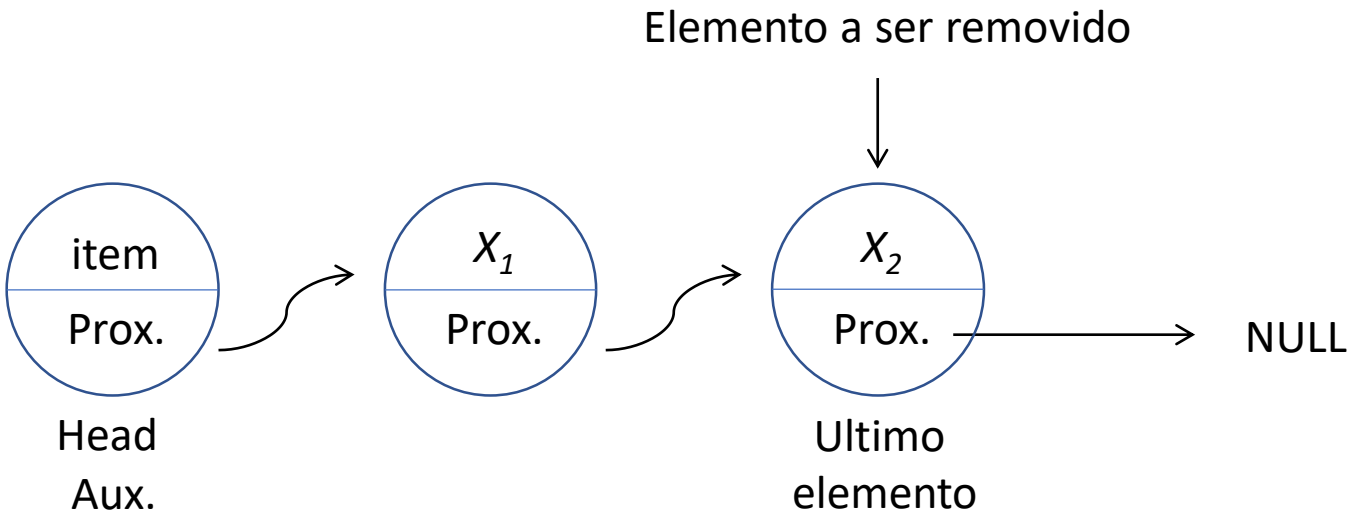
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

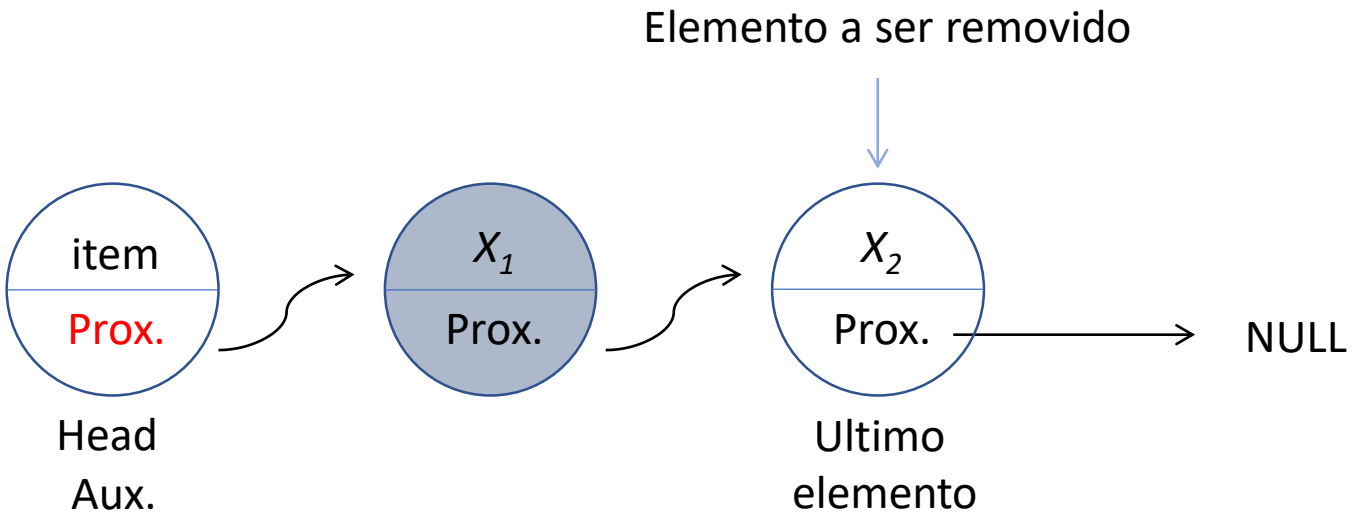



```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Observe que o objeto “primeiro” não armazena o primeiro elemento da lista, e sim é uma referência à célula *head* que referencia o primeiro elemento.

Implementação de Listas por meio de Estruturas Auto Referenciadas

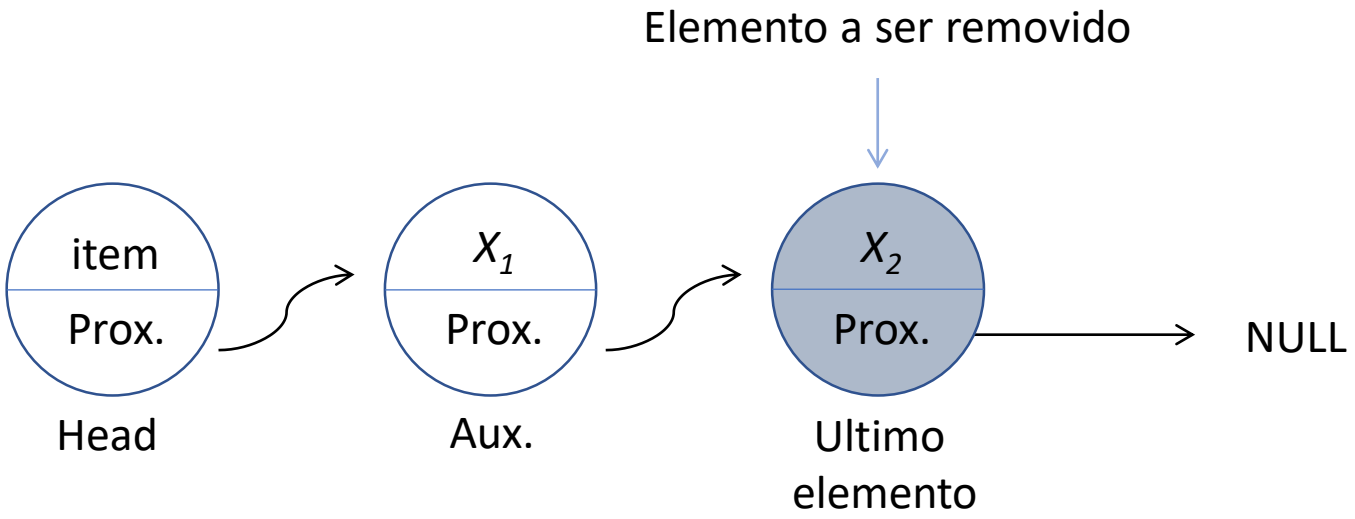
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

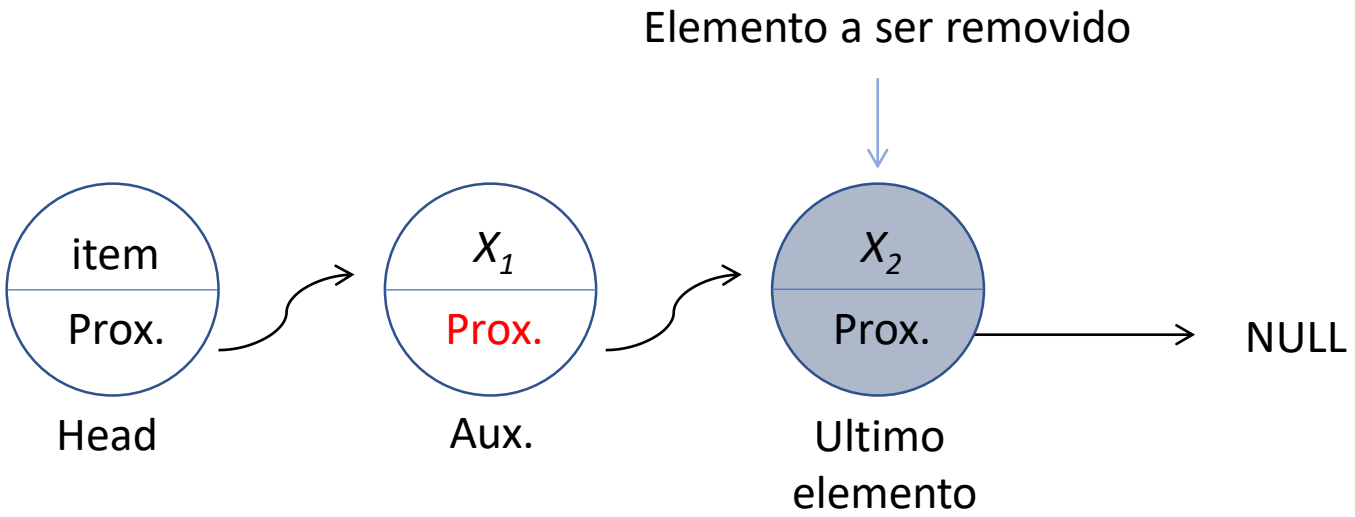
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

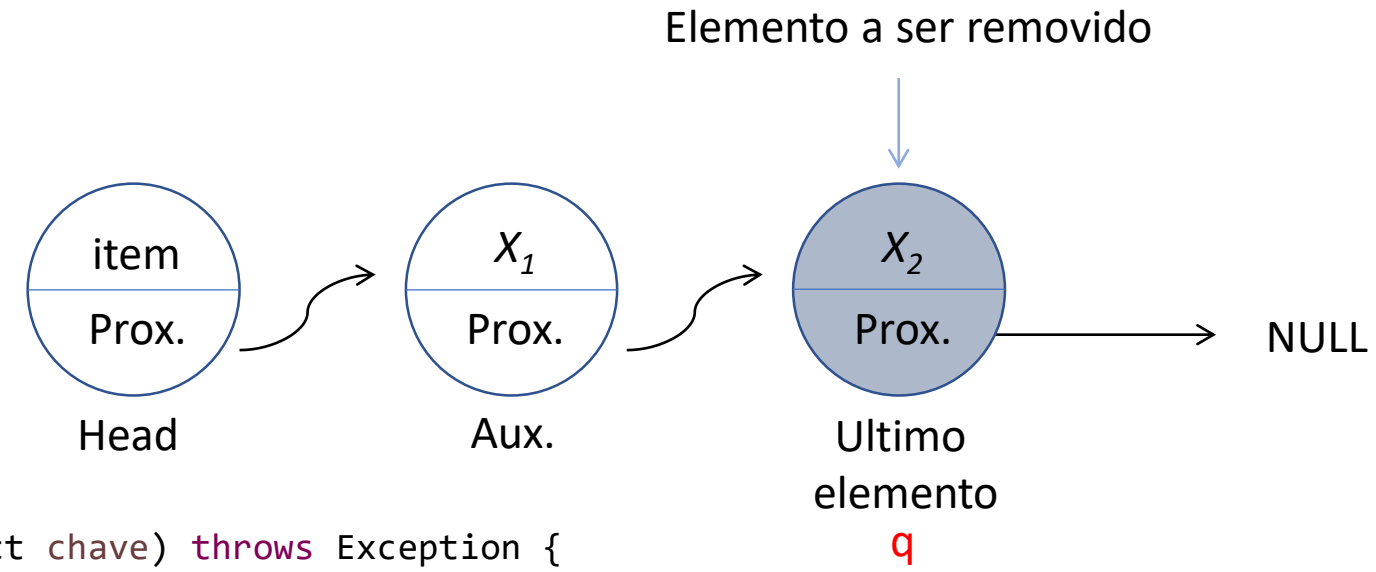
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

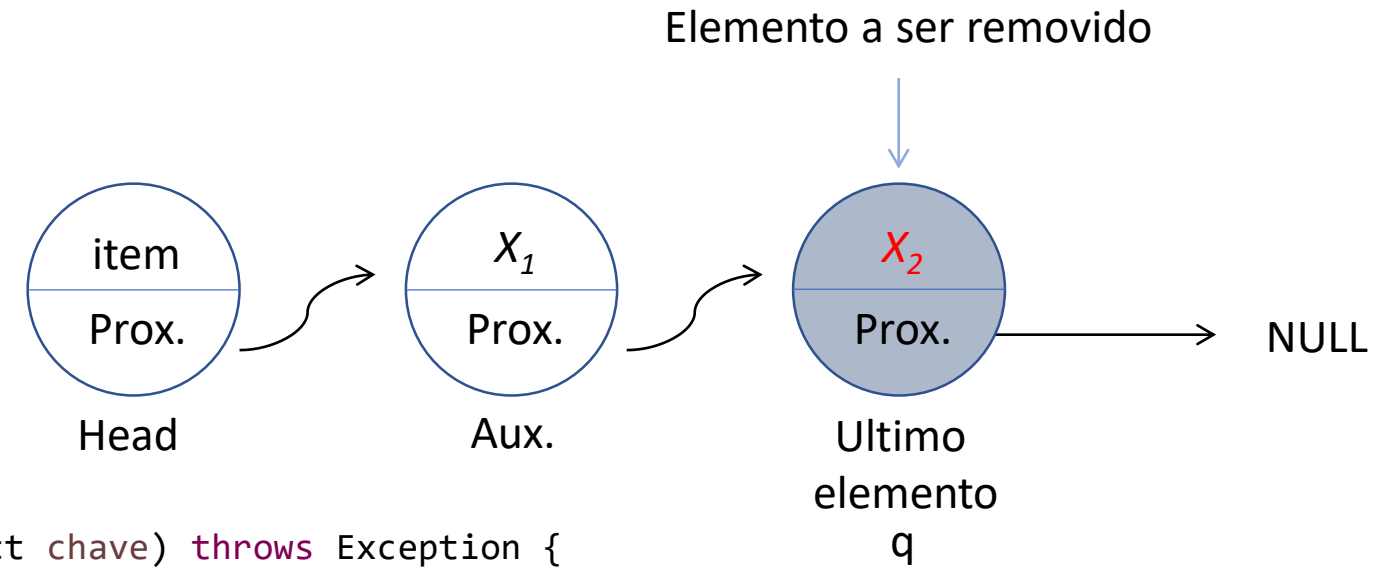
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

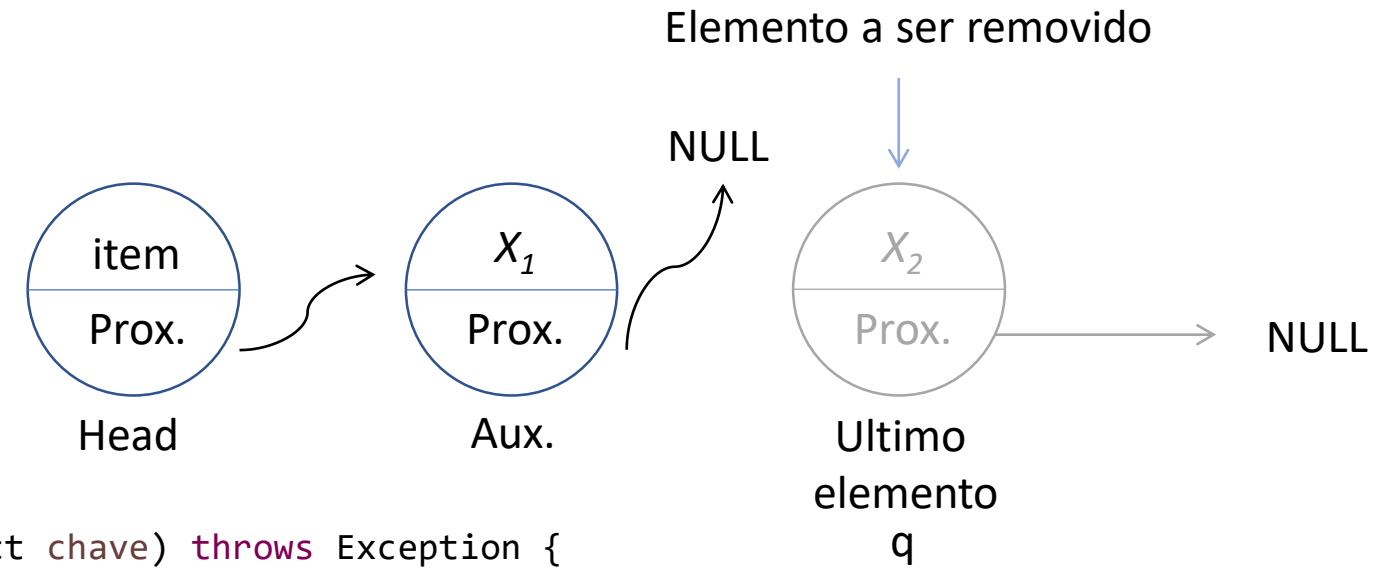



```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Recupera o objeto a ser removido

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

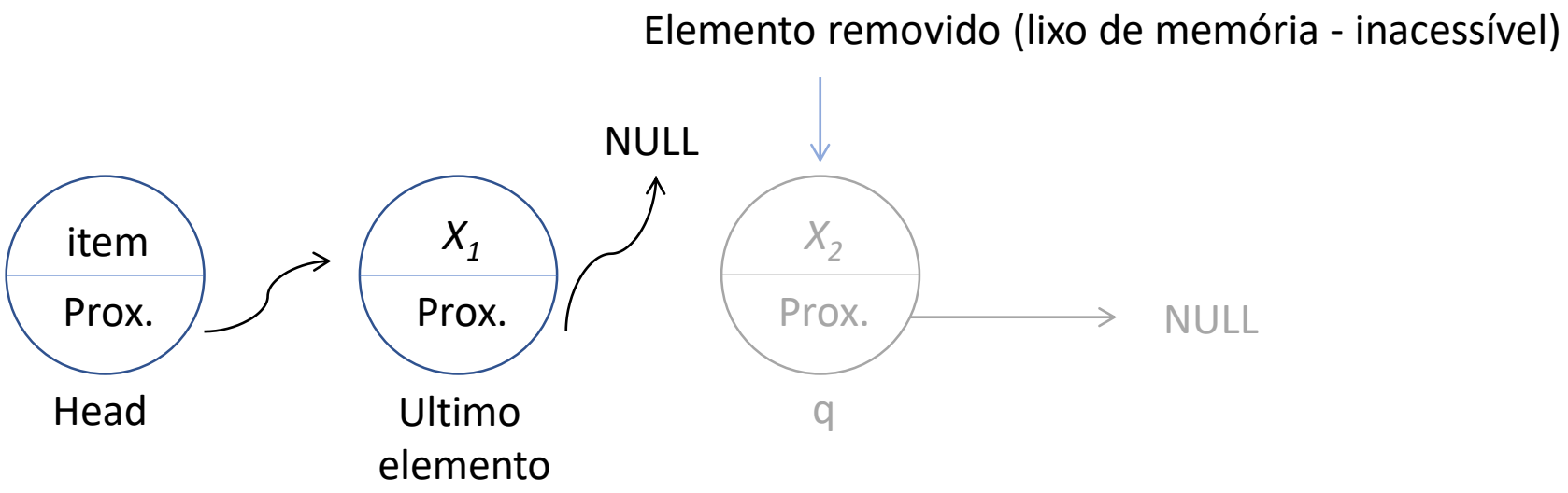



```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Faça aux.prox referenciar o próximo elemento da lista. Neste caso nulo, pois não há mais elementos.

Implementação de Listas por meio de Estruturas Auto Referenciadas

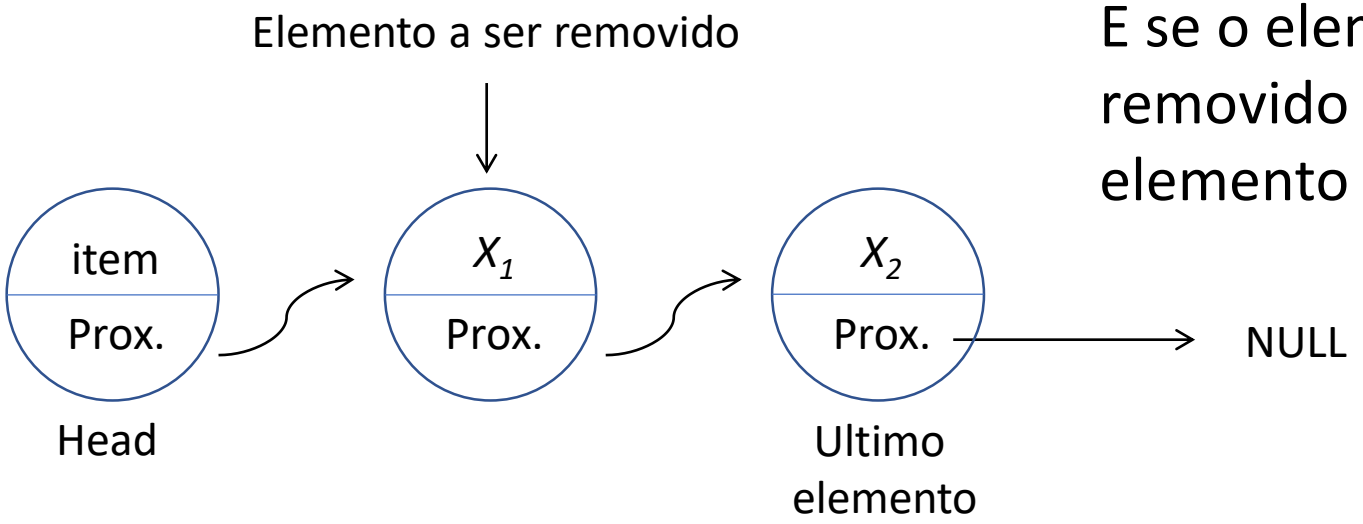
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

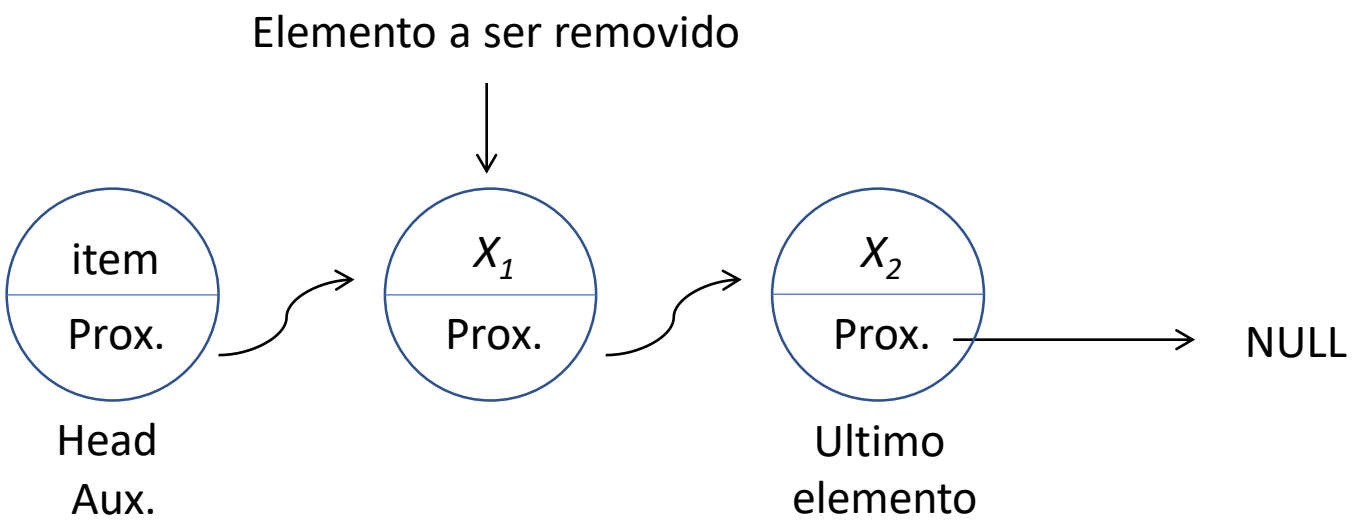


E se o elemento a ser removido não for o último elemento da lista?

```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

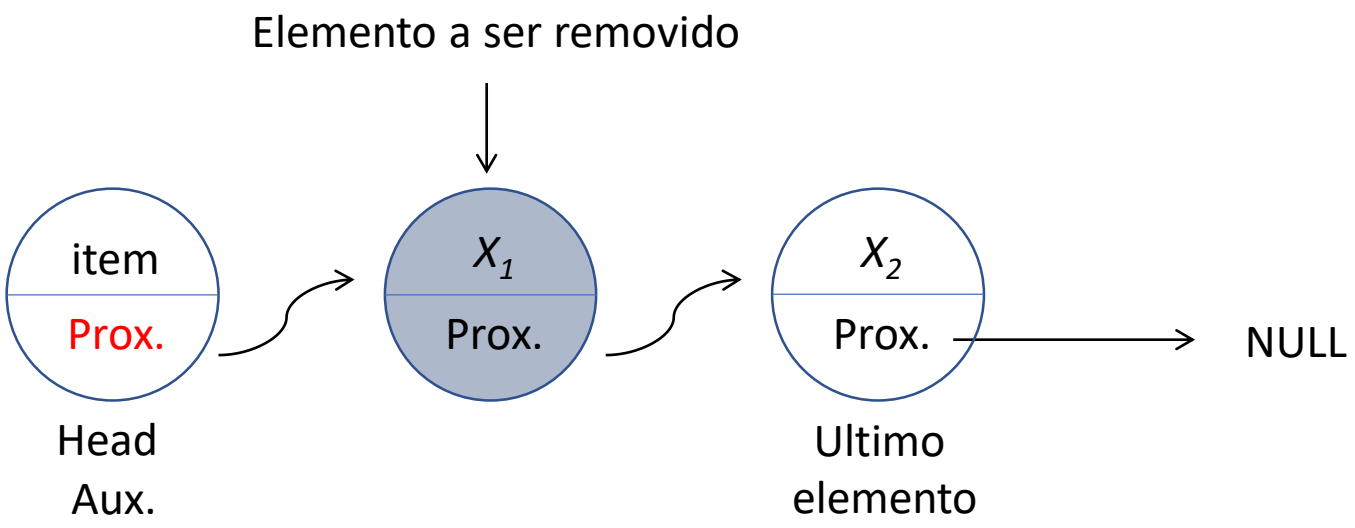



```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Observe que o objeto “primeiro” não armazena o primeiro elemento da lista, e sim é uma referência à célula *head* que referencia o primeiro elemento.

Implementação de Listas por meio de Estruturas Auto Referenciadas

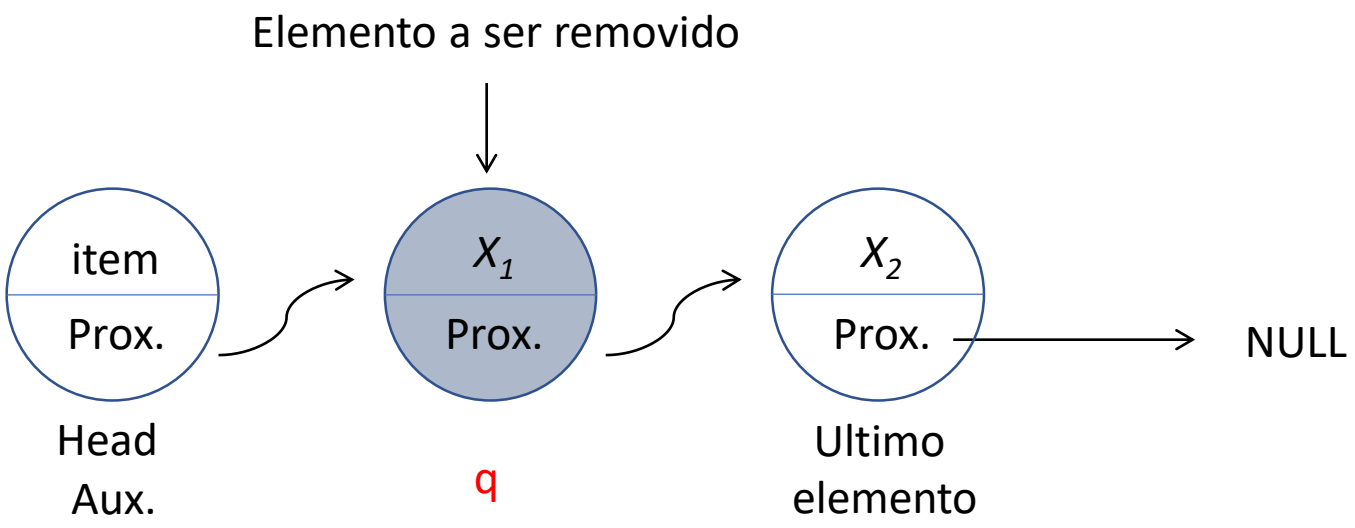
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

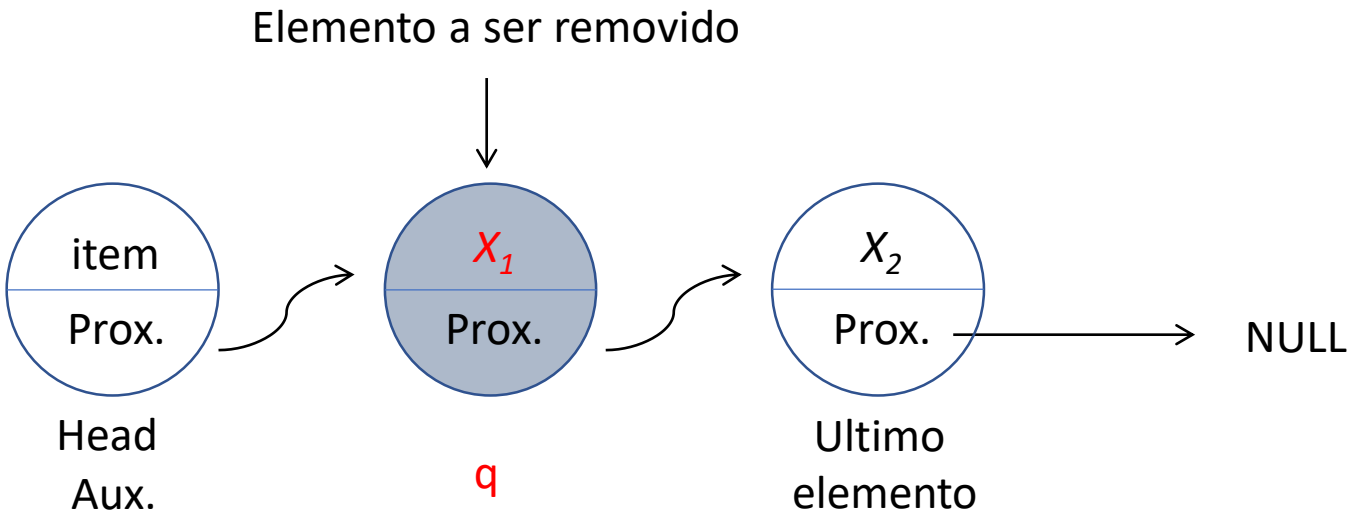
- Remoção de um elemento na lista:




```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

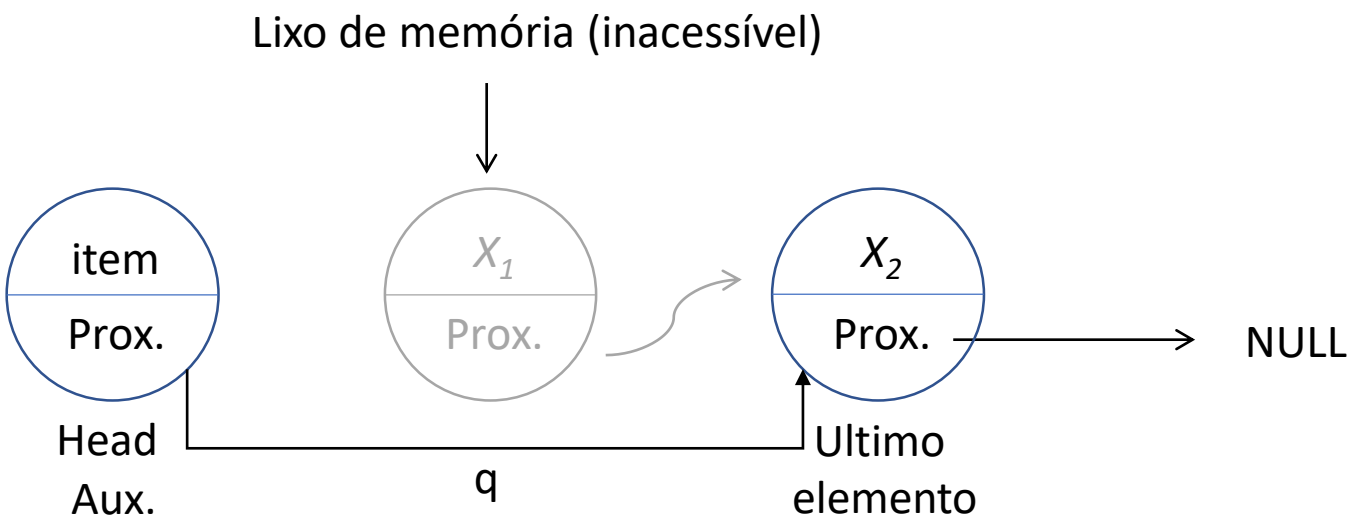



```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Recupera o objeto a ser removido

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

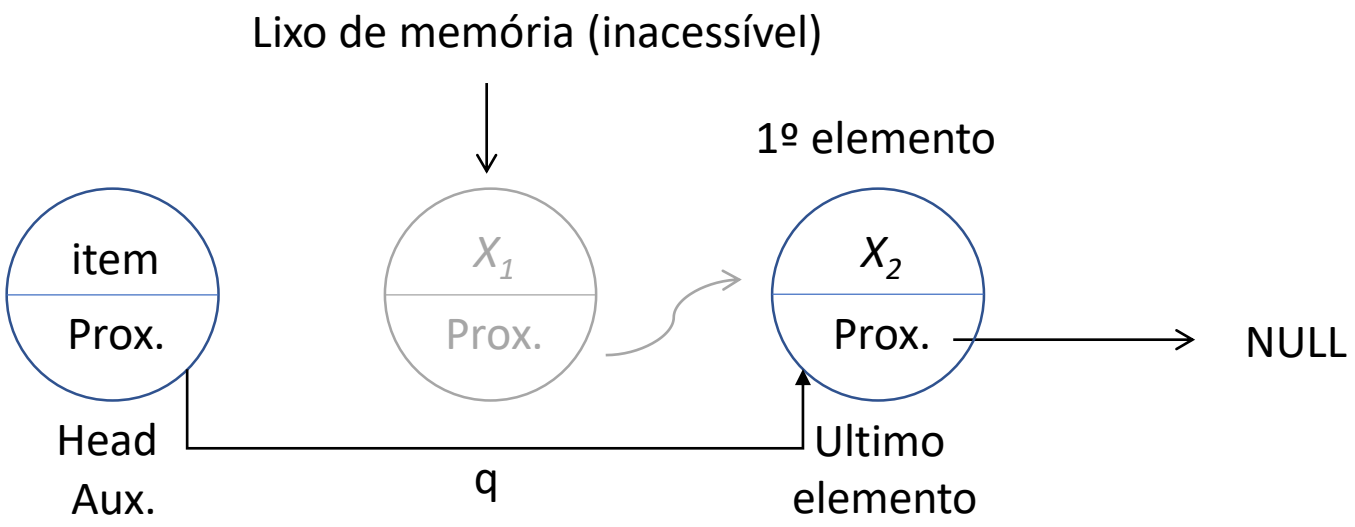



```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Faça aux.prox referenciar o próximo elemento da lista. Neste caso, o último elemento.

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

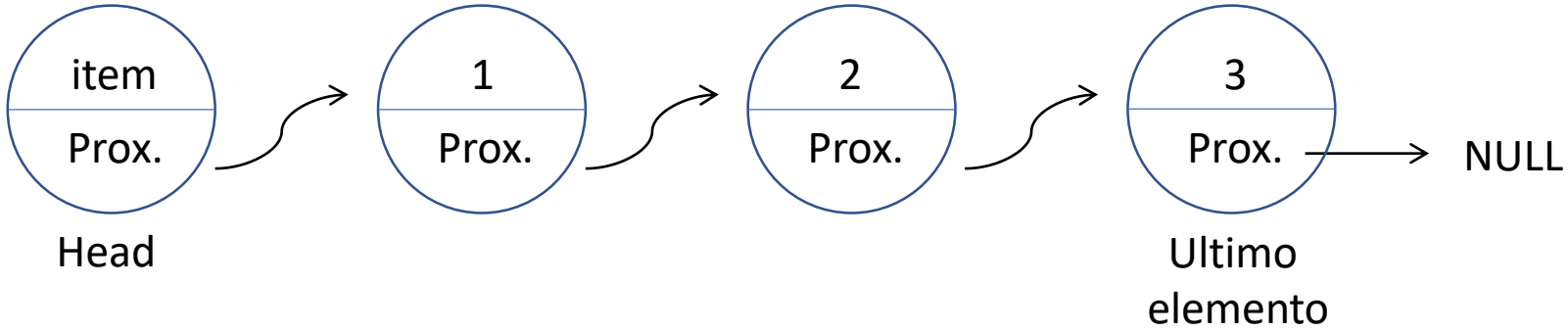



```
public Object retira (Object chave) throws Exception {
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux; return item;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

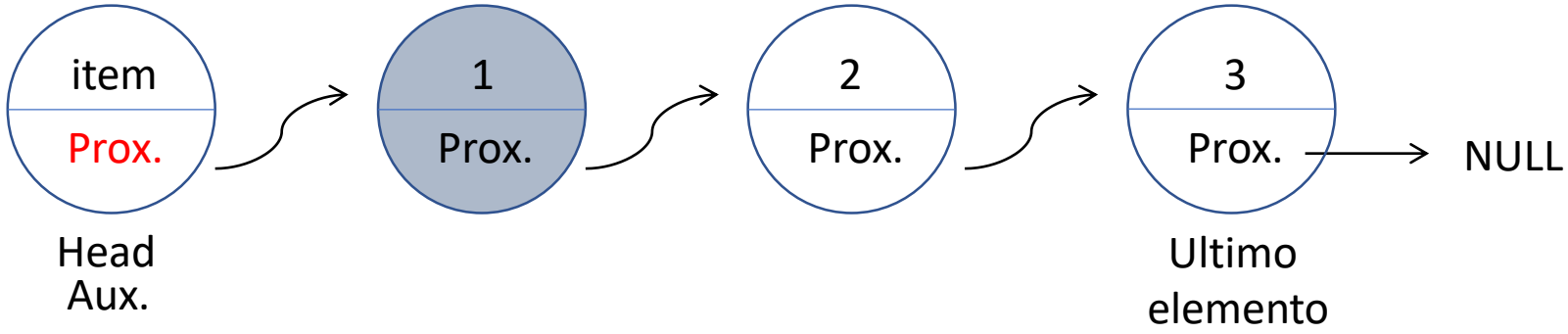



```
public Object pesquisa (Object chave) {
    Celula aux = this.primeiro;
    while (aux.prox != null) {
      if (aux.prox.item.equals (chave)) return aux.prox.item;
      aux = aux.prox;
    }
    return null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

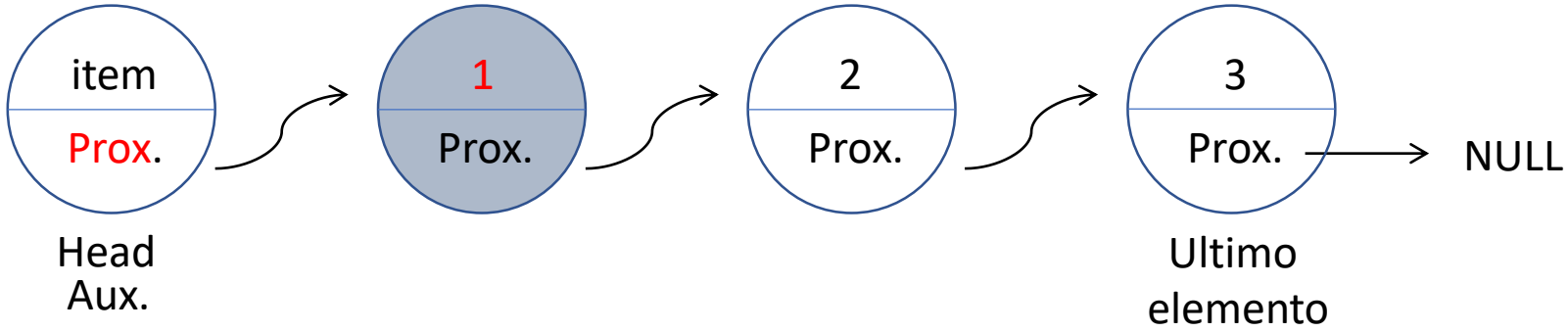



```
public Object pesquisa (Object chave) {
    Celula aux = this.primeiro;
    while (aux.prox != null) {
      if (aux.prox.item.equals (chave)) return aux.prox.item;
      aux = aux.prox;
    }
    return null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

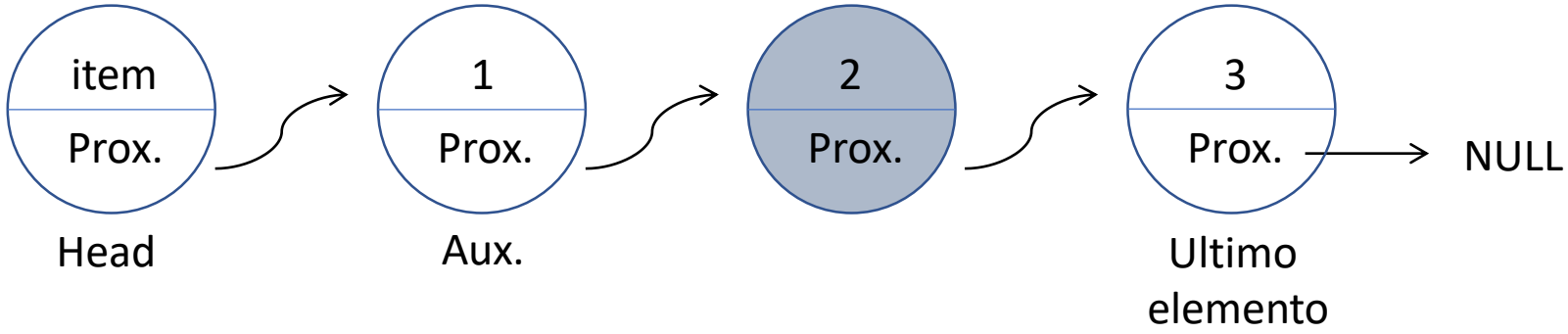



```
public Object pesquisa (Object chave) {
    Celula aux = this.primeiro;
    while (aux.prox != null) {
      if (aux.prox.item.equals (chave)) return aux.prox.item;
      aux = aux.prox;
    }
    return null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

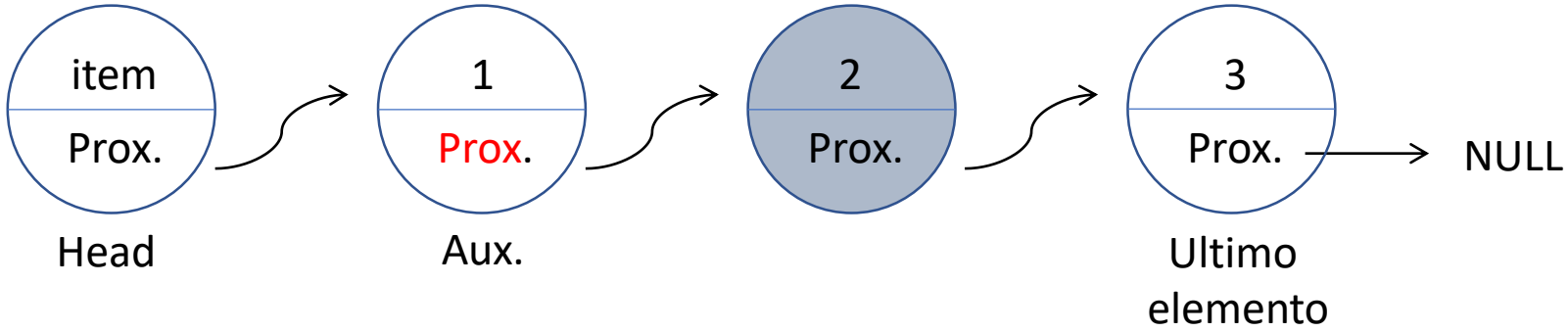



```
public Object pesquisa (Object chave) {
    Celula aux = this.primeiro;
    while (aux.prox != null) {
      if (aux.prox.item.equals (chave)) return aux.prox.item;
      aux = aux.prox;
    }
    return null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

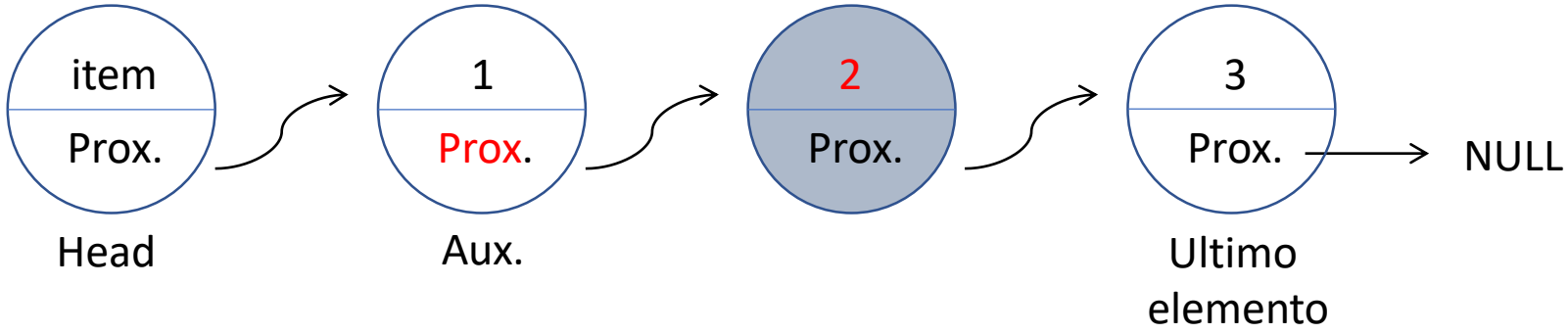



```
public Object pesquisa (Object chave) {
    Celula aux = this.primeiro;
    while (aux.prox != null) {
      if (aux.prox.item.equals (chave)) return aux.prox.item;
      aux = aux.prox;
    }
    return null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);




```
public Object pesquisa (Object chave) {
    Celula aux = this.primeiro;
    while (aux.prox != null) {
      if (aux.prox.item.equals (chave)) return aux.prox.item;
      aux = aux.prox;
    }
    return null;
  }
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- A classe Lista contém:
  - uma referência para a célula *head*;
  - uma referência para a última célula da lista; e
  - uma referência para armazenar a posição corrente na lista.

Filas usando Estruturas Auto Referenciadas

```
public class Lista {
  private static class Celula {
     Object item;
     Celula prox;
  }
  private Celula primeiro, ultimo, pos;

  public Lista () {
    this.primeiro = new Celula (); this.pos = this.primeiro;
    this.ultimo = this.primeiro; this.primeiro.prox = null;
  }
  public Object pesquisa (Object chave) {
    if (this.vazia () || chave == null) return null;
    Celula aux = this.primeiro;
    while (aux.prox != null) {
      if (aux.prox.item.equals (chave))
         return aux.prox.item;
      aux = aux.prox;
    }
    return null;
  }
  public void insere (Object x) {
    this.ultimo.prox = new Celula (); this.ultimo = this.ultimo.prox;
    this.ultimo.item = x; this.ultimo.prox = null;
  }
```

Observe que a classe Lista contém a classe “Celula”. Esta classe possui um objeto item (tipo genérico) e um objeto “prox” que é uma referência para a própria classe “Celula”.

Objetos do tipo “Celula”, sempre irão fazer referência ao tipo célula.

Criando uma Lista vazia. Observe que “primeiro” é igual a “ultimo”

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Filas usando Estruturas Auto Referenciadas

```
//Insere antes do primeiro item da lista
  public void inserePrimeiro (Object item) {
    Celula aux =  this.primeiro.prox;
    this.primeiro.prox = new Celula ();
    this.primeiro.prox.item = item;
    this.primeiro.prox.prox = aux;
  }
  public Object retira (Object chave) throws Exception {
    if (this.vazia () || (chave == null))
      throw new Exception ("Erro: Lista vazia ou chave invalida");
    Celula aux = this.primeiro;
    while (aux.prox != null && !aux.prox.item.equals (chave))
      aux = aux.prox;
    if (aux.prox == null) return null;
    Celula q = aux.prox; Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux;
    return item;
  }
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Filas usando Estruturas Auto Referenciadas

```
public Object retiraPrimeiro () throws Exception {
    if (this.vazia ()) throw new Exception ("Erro: Lista vazia");
    Celula aux = this.primeiro; Celula q = aux.prox;
    Object item = q.item; aux.prox = q.prox;
    if (aux.prox == null) this.ultimo = aux;
    return item;
  }
  public boolean vazia () { return (this.primeiro == this.ultimo); }
  public void imprime () {
    Celula aux = this.primeiro.prox;
    while (aux != null) {
      System.out.println (aux.item.toString ()); aux = aux.prox;
    }
  }
  public boolean estaNaLista (Object chave) {
    Object obj = this.pesquisa (chave);
    return (obj != null);
  }
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Filas usando Estruturas Auto Referenciadas

```
public Object primeiro () {
    //Retorna o primeiro item da lista
    this.pos = this.primeiro; return this.proximo ();
  }

  public Object proximo () {
    //Retorna o próximo item da lista
    this.pos = this.pos.prox;
    if (this.pos == null) return null;
    else return this.pos.item;
  }

}
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Teste da Lista

```
public class TestaLista {
  public static void main (String[] args) {
    Lista lista = new Lista();
    Integer vetor[] = new Integer[1000];
    Integer n = null;
    int i, j, k, max;
    java.util.Random random = new java.util.Random();
    max = 10;
    try {
      // Gera uma permutacao aleatoria de chaves entre 0 e max - 1
      for (i = 0; i < max; i++)
        vetor[i] = new Integer(i);
      for (i = 0; i < max; i++) {
        k = Math.abs(random.nextInt()) % max;
        j = Math.abs(random.nextInt()) % max;
        n = vetor[k];
        vetor[k] = vetor[j];
        vetor[j] = n;
      }
```

Utilize a classe ao lado para testar a Lista por vetor e com estruturas auto referenciadas

Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Teste da Lista

```
// Insere cada chave na lista
      for (i = 0; i < max; i++) {
        lista.insere(vetor[i]);
        System.out.println("Inseriu " + vetor[i].toString());
      }
      lista.imprime();
      // Pesquisa cada chave da lista
      for (i = 0; i < max; i++) {
        n = (Integer) lista.pesquisa(vetor[i]);
        System.out.println("Item: " + n.toString());
      }
      // Pesquisa sem sucesso
      Object item = lista.pesquisa(new Integer(100));
      if (item == null)
        System.out.println("Item nao encontrado");
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Teste da Lista

```
// Retira cada chave da lista
      for (i = 0; i < max; i++) {
        n = (Integer) lista.retira(vetor[i]);
        System.out.println("Retirou: " + n.toString());
      }
      n = (Integer) lista.retiraPrimeiro();
    } catch (Exception e) {
      System.out.println(e.getMessage());
    }
    lista.imprime();
  }
}
```


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Vantagem:
  - Permite inserir ou retirar itens do meio da lista a um custo constante
    - importante para lista de elementos ordenados.
  - Bom para aplicações em que não existe previsão sobre o crescimento da lista
    - o tamanho máximo da lista não precisa ser definido.

- Desvantagens:
  - utilização de memória extra para armazenar as referências

Exercícios

- Implementar classe Lista (por vetor) e Lista (encadeada), e implementar, para cada tipo de lista, um programa que:
  - Construa a lista vazia;
  - Insira novos elementos na lista;
  - Pesquisa um elemento;
  - Retire um elemento.
- Para a Lista (por vetor), implemente uma função para realocar o tamanho do vetor de itens da lista.
- Implemente uma função para Lista (Lista encadeada) para inserir um elemento em qualquer posição da lista ordenada.
  - Crie a lista com os elementos: 1, 2, 3, 5, 6.
  - Insira o elemento 4 de forma ordenada na lista.

Listas Duplamente Ligadas

- As Operações de remoção de uma elemento indica um problema inerente às listas ligadas.

- As células das listas contém apenas a referência para os seus sucessores.

- Não existe um acesso imediato aos seus predecessores.

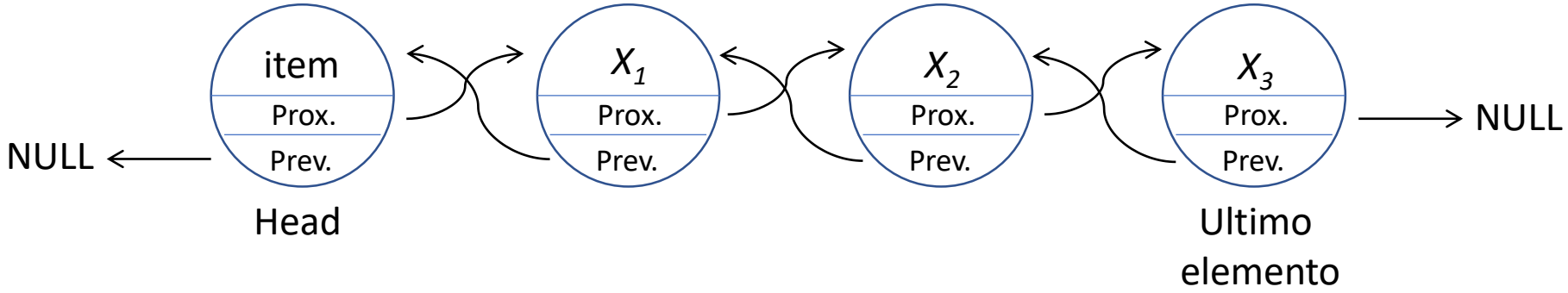
Listas Duplamente Ligadas

- Para remover o último elemento da lista deve-se percorrer toda a lista até encontrar o penúltimo elemento.

- Para listas longas e muita utilização da remoção do ultimo elemento, podem impactar no processamento rápido da lista.

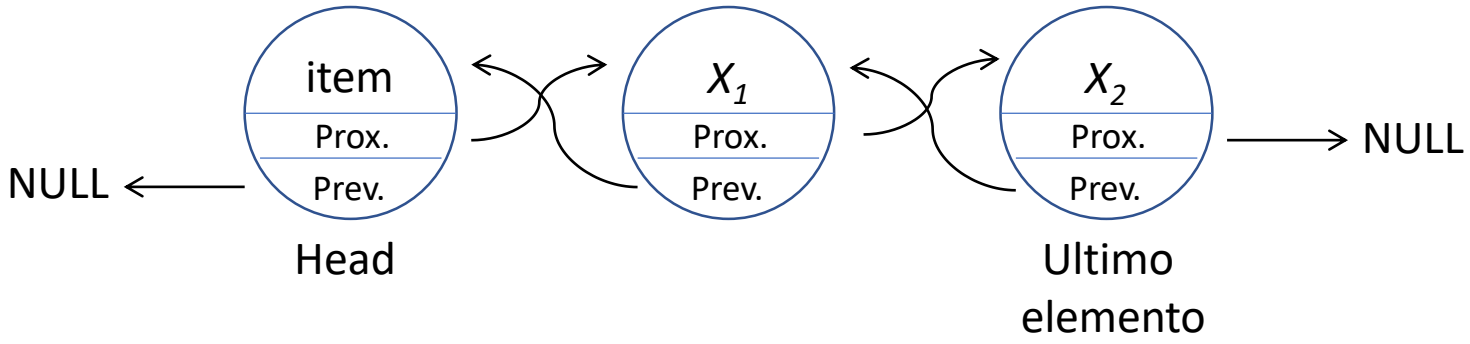
Listas Duplamente Ligadas

- As listas duplamente ligadas (ou duplamente encadeadas) cada célula da lista contem duas referências:
  - Uma para o sucessor (igual nas listas ligadas)
  - Outra para o predecessor.



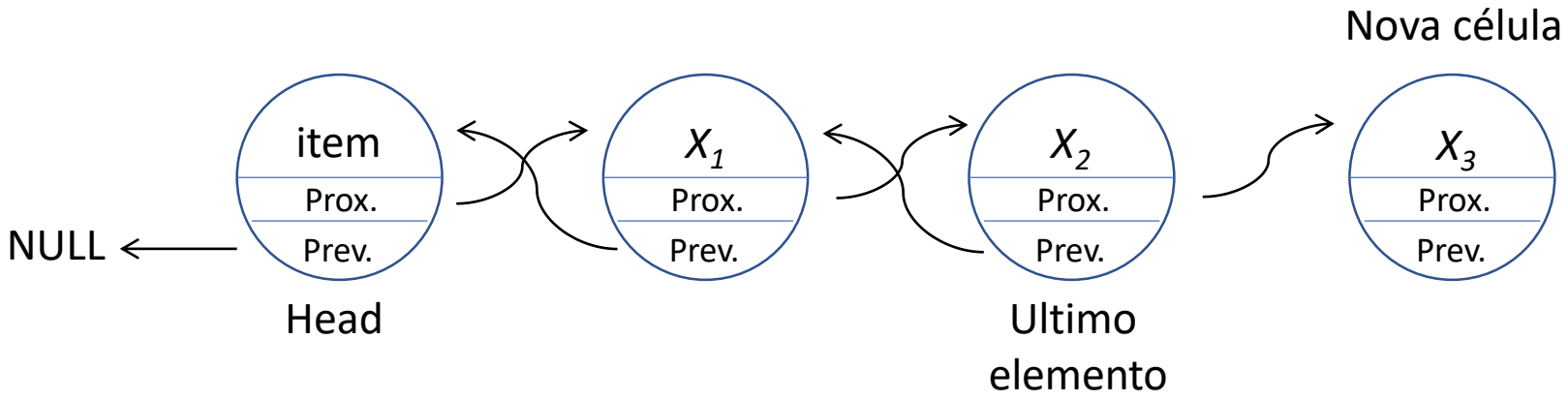
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - Lista com dois elementos:



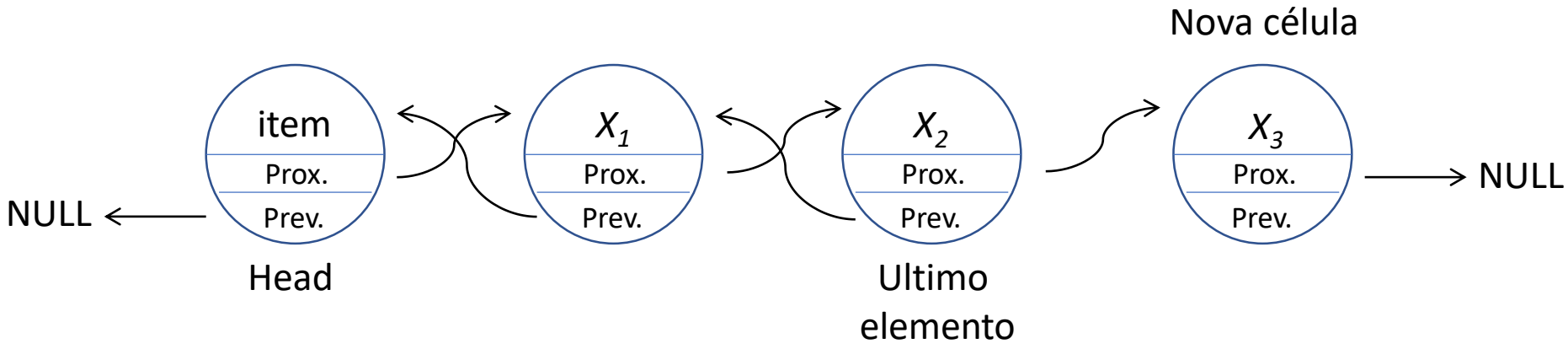
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - Uma nova célula é criada e seus três atributos são inicializados:



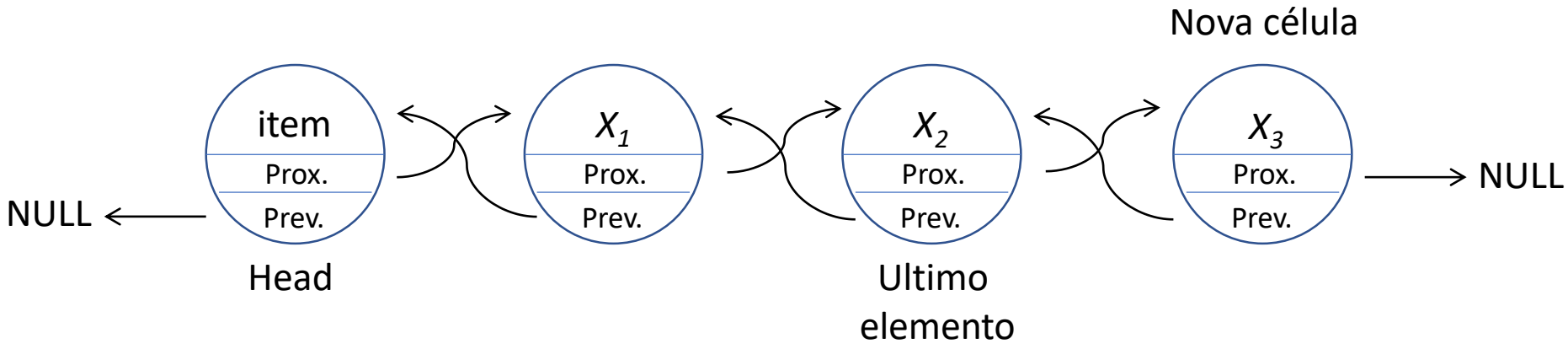
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - “prox” da nova célula referencia nulo:



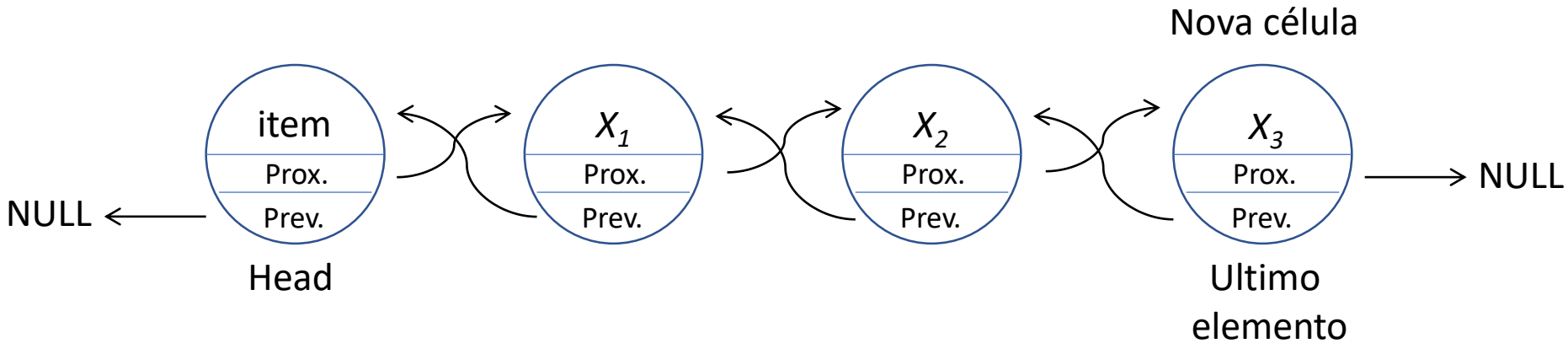
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - “prev” referencia o (até então) último elemento da lista:



Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - “ultimo” é ajustado para o novo elemento inserido:

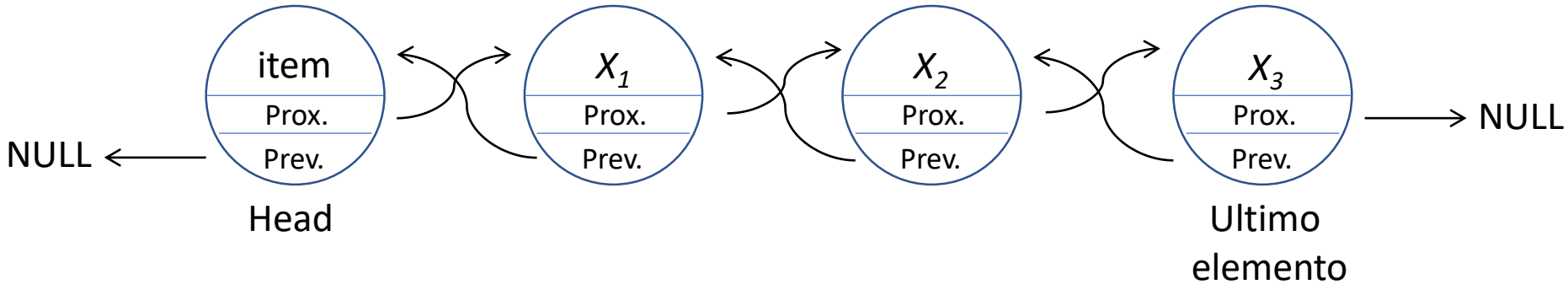


Listas Duplamente Ligadas

- Remoção do último elemento da lista:
  - Remover o último elemento de uma lista duplamente ligada é simples pois há acesso direto do ultimo elemento ao seu predecessor, e nenhum laço é necessário para remover o último elemento

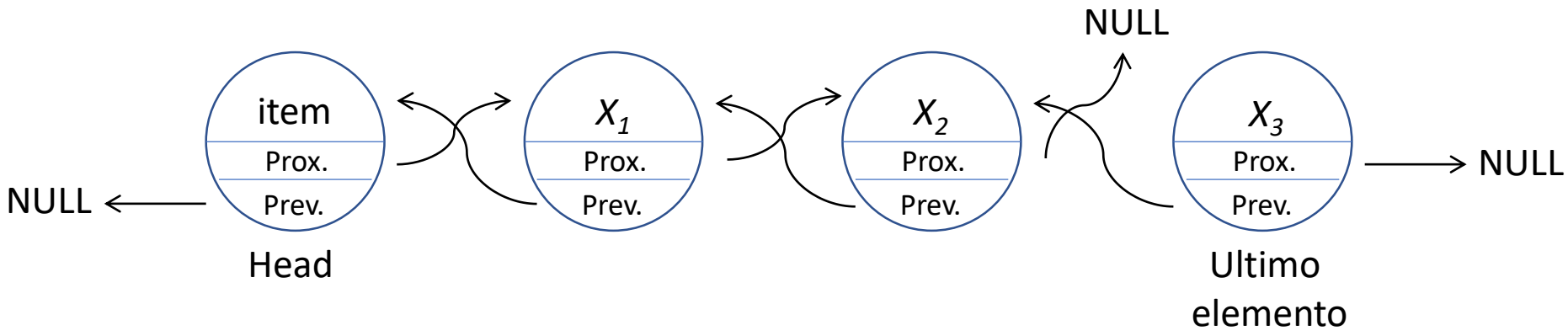
Listas Duplamente Ligadas

- Remoção do ultimo elemento da lista:
  - Não é necessário percorrer a lista.



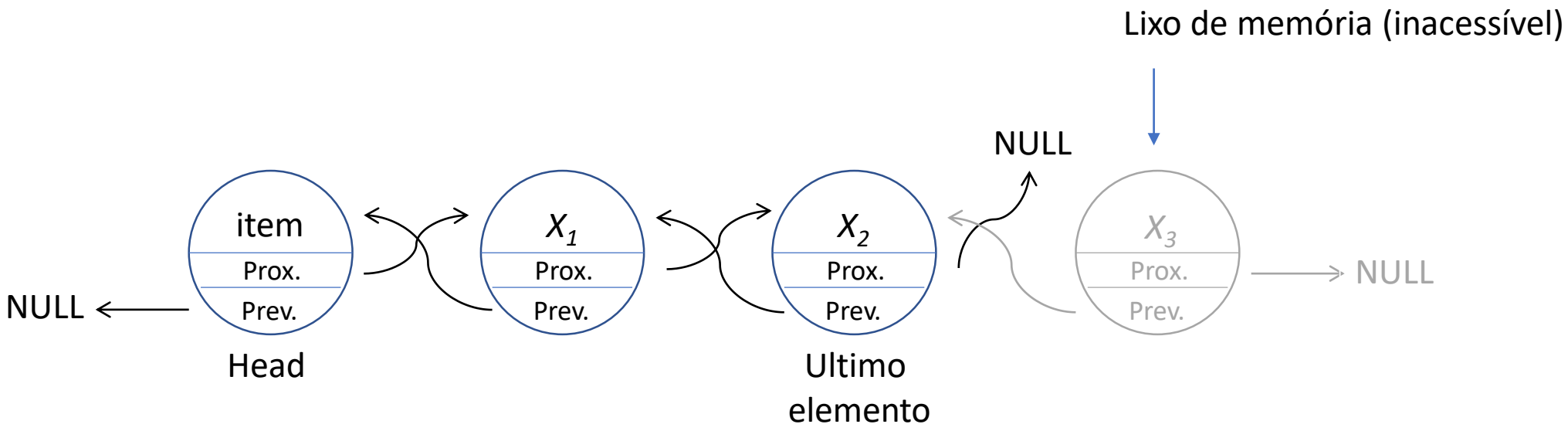
Listas Duplamente Ligadas

- Remoção do ultimo elemento da lista:
  - Após recuperar o item a ser removido, “prox” do predecessor passa a referenciar NULL



Listas Duplamente Ligadas

- Remoção do ultimo elemento da lista:
  - Ultimo elemento é ajustado para o predecessor do elemento a ser removido.



Listas Duplamente Ligadas

- Devido a imediata acessibilidade ao ultimo elemento, tanto a operação de inserção quanto a remoção do ultimo elemento são executadas sem a necessidade de percorrer a lista.

Listas Circulares

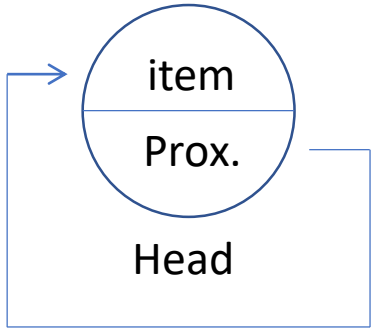
- São listas na qual as células formam um anel:
  - A lista é finita; e
  - Cada célula tem um sucessor.
- O sucessor do último elemento é o primeiro elemento da lista.
- Uma mudança na lista linear:
  - O campo *prox* do ultimo elemento referencia o primeiro elemento, ou invés de nulo.

Listas Circulares

- Neste caso, pode-se usar somente uma referência permanente: o primeiro elemento da lista, ao contrário da lista ligada que utilizava duas: primeiro (*head*) e ultimo.

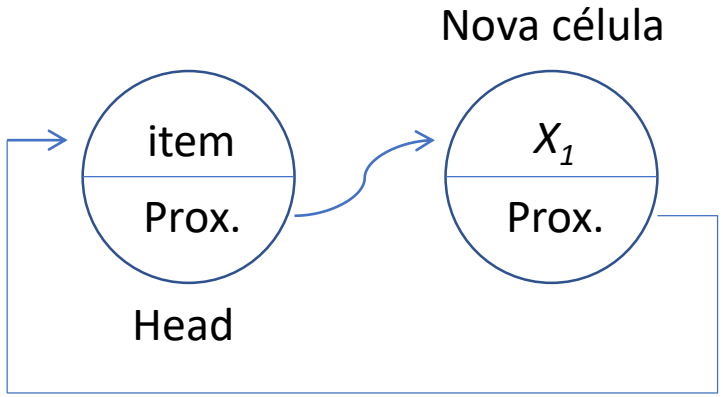
Listas Circulares

- Lista circular vazia (note que não há mais a referência ao ultimo elemento):



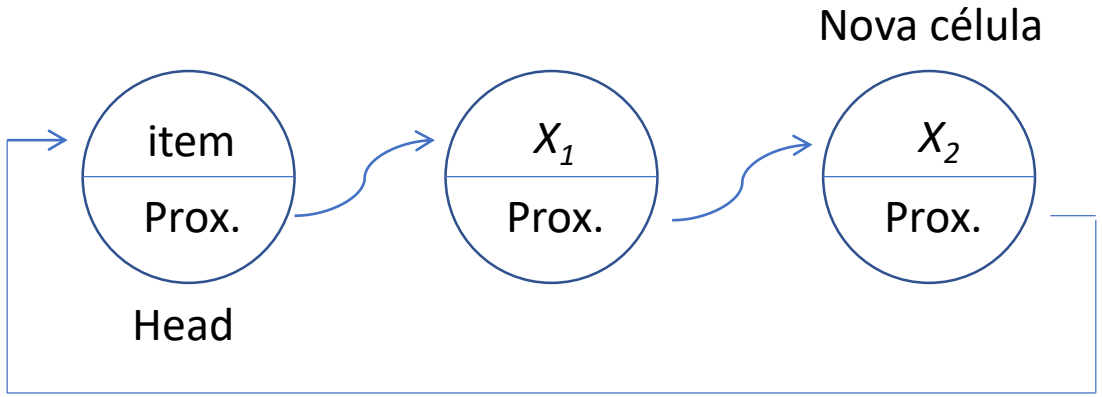
Listas Circulares

- Inserção de um novo elemento na lista.



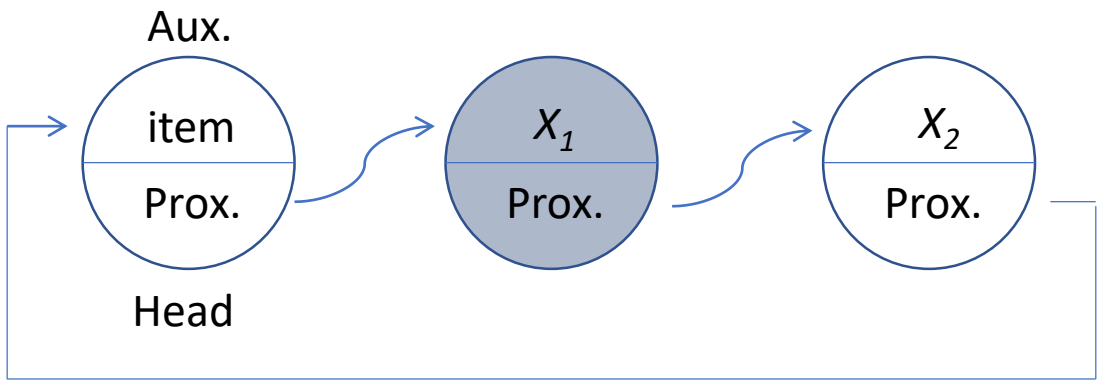
Listas Circulares

- Inserção de um novo elemento na lista.



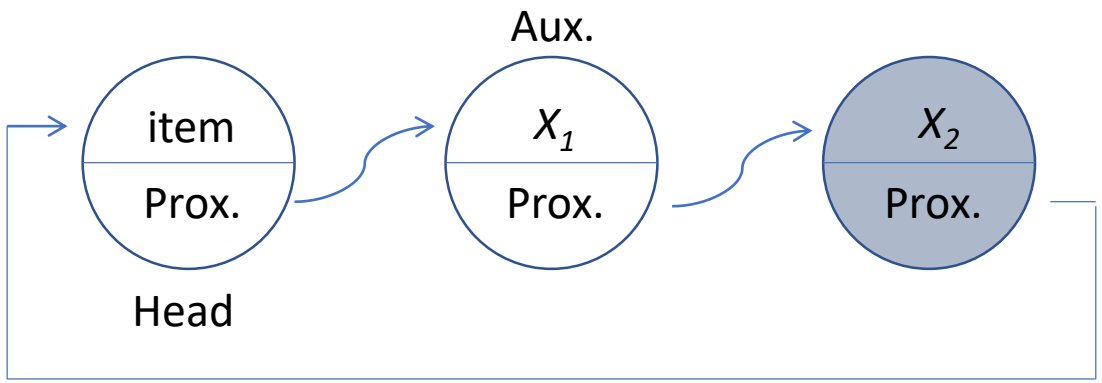
Listas Circulares

- Removendo o ultimo elemento da lista.



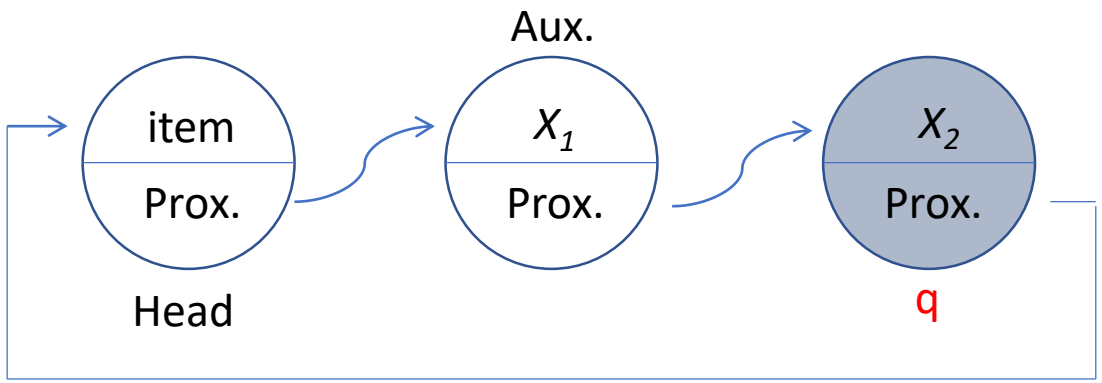
Listas Circulares

- Removendo o ultimo elemento da lista.



Listas Circulares

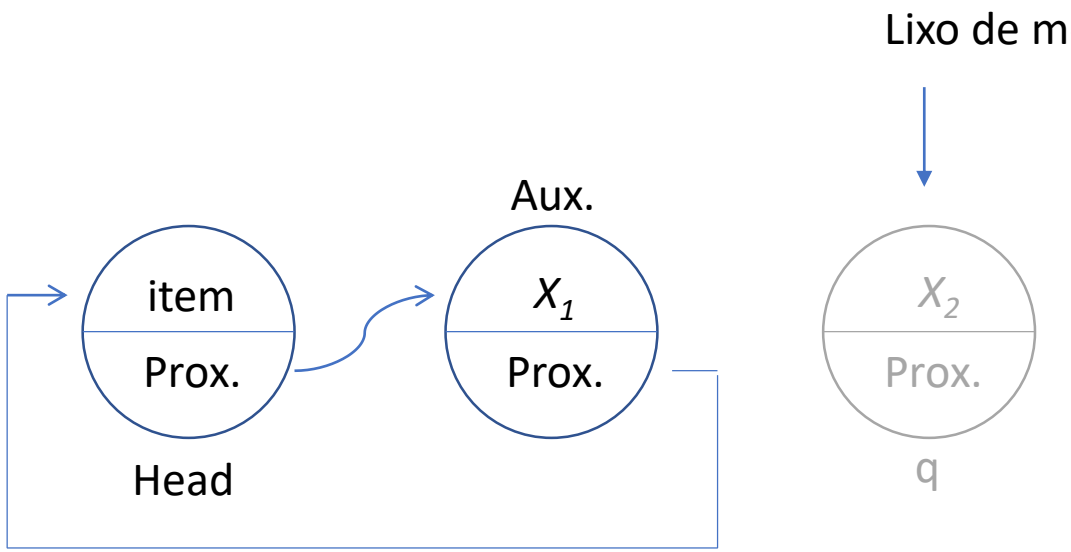
- Removendo um elemento da lista.



Recupera o elemento
a ser removido

Listas Circulares

- Removendo um elemento da lista.



Ao invés de referenciar nulo, passa a
referenciar o primeiro elemento (próximo elemento).

Listas Circulares

- Como nas listas lineares, para remover o ultimo elemento é necessário percorrer toda a lista.

- No entanto, com listas circulares é possível acessar qualquer elemento de qualquer ponto da lista (uma vez que a partir do último é possível acessar o primeiro).

Exercícios

- Baseado na implementação da lista ligada (lista auto referenciada ou encadeada) apresentada em aula, implemente as listas Duplamente Encadeada e a Lista Circular e suas operações básicas.
- Para a lista Duplamente Encadeada implemente operações que:
  - Insere novo elemento no meio da lista (ordenada);
  - Remove ultimo elemento;

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).